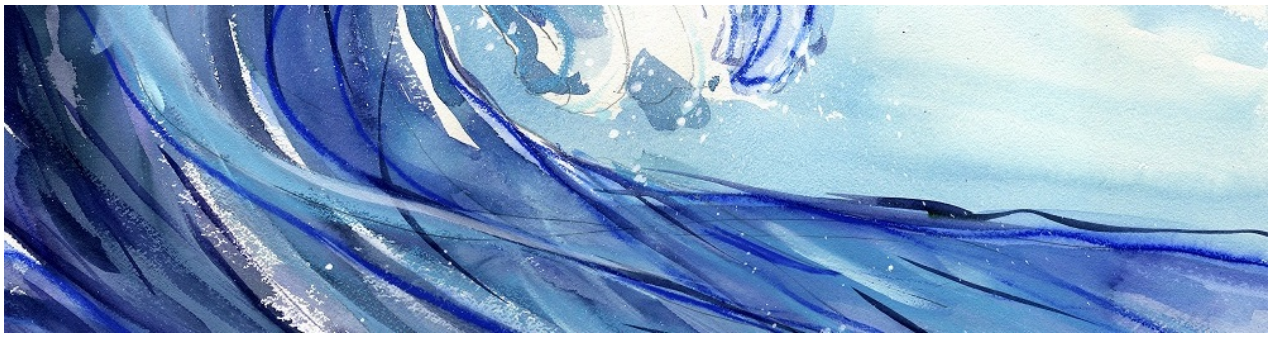
INFORMATION IMAGES LEARNING

An Introduction to Data Sciences

Gabriel Peyré

CNRS & DMA
École Normale Supérieure, PSL University

September 9, 2026



An introduction to the mathematical ideas behind representing, processing and learning from data.

https://mathematical-tours.github.io/book-basics/

About this book

This book offers five introductions to mathematical ideas that help us represent, process, and learn from data. Concrete examples provide a starting point, while precise statements explain what an algorithm can and cannot guarantee.

- Chapter 1 introduces binary coding and Shannon's entropy bound for lossless compression.
- Chapter 2 treats an image as an array of numbers and develops quantization, filtering, edge detection, and color transformations.
- Chapter 3 connects sampling and sparse approximation to the regularization of inverse problems and compressed sensing.
- Chapter 4 develops optimal transport, from finite assignments to the comparison and interpolation of distributions.
- Chapter 5 explains discriminative and generative neural networks, their training objectives, and their connection with transport.

The first two chapters require elementary algebra and, for information theory, logarithms and basic probabilities. The later chapters use vectors, matrices, derivatives, and probability distributions. Their more advanced results are stated with their assumptions; the surrounding examples can be read before studying those results in detail.

Each chapter can be read separately. Reading them in order reveals useful connections: entropy explains coding, transforms make signals compressible, regularization selects reconstructions, and transport compares the distributions that a generative model seeks to learn. A common notation and short chapter introductions support both reading paths.

The English and French editions contain the same five chapters. Complete books, individual chapters, and the sources are available at

https://mathematical-tours.github.io/book-basics/

For a more extensive treatment, see *Mathematical Foundations of Data Sciences*, available from https://mathematical-tours.github.io/book/.

Acknowledgements. I thank Marie-Noëlle Peyré, Gwenn Guichaoua, François Béguin, Gérard Grancher, Aurélien Djament, François Sauvageot, Charles Dossal, Jalal Fadili, Samuel Vaiter, Stéphane Seuret, Vincent Beck, Sébastien Racanière, Vincent Barra, and the anonymous reviewer of the original sparsity article for their careful reading and corrections. The flower photograph is by Maïtine Bergounioux.

Gabriel Peyré
CNRS & DMA, École Normale Supérieure, PSL

Contents

About this book ii

1 Claude Shannon and data compression 1
- 1.1 Digital data and coding . . . 1
- 1.2 Encoding and decoding . . . 1
- 1.3 Entropy and the Shannon bound . . . 3

2 Image processing 7
- 2.1 Pixels and grayscale values . . . 7
- 2.2 Storing an image . . . 7
- 2.3 Removing noise . . . 9
- 2.4 Detecting edges . . . 10
- 2.5 Color images . . . 10
- 2.6 Changing brightness and contrast . . . 12
- 2.7 Images as matrices . . . 13

3 Sparsity, inverse problems and compressed sensing 15
- 3.1 From continuous signals to samples . . . 15
- 3.2 Nonlinear approximation and compression . . . 16
- 3.3 Inverse problems and regularization . . . 17
- 3.4 Compressed sensing . . . 19

4 Optimal transport 21
- 4.1 Monge's problem . . . 21
- 4.2 Kantorovich's relaxation . . . 22
- 4.3 Measures and transport distances . . . 24
- 4.4 Applications . . . 25

5 Neural networks 27
- 5.1 Discriminative networks . . . 27
- 5.2 Generative networks . . . 30

Glossary 33

References 34

CHAPTER 1

Claude Shannon and data compression

How many bits are needed to store a message without losing information? Shannon's theory relates this question to the probabilities of the symbols in the message. Starting from a small grayscale image, we introduce binary codes, explain unambiguous decoding, and derive the role of entropy in compression.

1.1 Digital data and coding

Text, sound, images and video are stored and transmitted as sequences of bits: binary digits 0 and 1. Before this binary representation is chosen, the data are described by *symbols*: letters in a text, for instance, or pixel values in an image. A lossless compression method seeks a compact binary representation of a sequence of symbols from which the original sequence can be recovered exactly.

Shannon's work [41, 42] established fundamental limits for representing and communicating information. It is useful to distinguish the operations involved:

- *Sampling* records a continuous signal at selected locations or times. *Quantization* then replaces its values by elements of a finite set.
- *Source coding*, the subject of this chapter, seeks a compact representation of the resulting symbols.
- *Channel coding* adds redundancy to detect or correct transmission errors. Protection against malicious alteration is a separate security problem.

Sampling and quantization may discard information. The coding studied here is lossless relative to the symbols that have already been acquired. This distinction matters when discussing the compression of photographs or sound recordings.

1.2 Encoding and decoding

1.2.1 A small grayscale image

A grayscale image is an array of pixels. For simplicity, consider four possible values: 0 for black, 1 for dark gray, 2 for light gray and 3 for white. The same ideas apply to a larger alphabet, such as the 256 values of an eight-bit grayscale image, and to the channels of a color image.

Figure 1.1 shows a flower and a magnified 5×5 patch. Reading its 25 values in the indicated order gives

$$(0,1,3,2,0,3,2,2,1,2,2,1,1,2,2,2,1,1,2,2,2,1,1,2,1). \tag{1.1}$$

1.2.2 Fixed-length coding

A *codeword* is the bit string assigned to a symbol. We require nonempty codewords unless the message length is known in advance. One possible assignment is

$$0 \mapsto 00, \qquad 1 \mapsto 01, \qquad 2 \mapsto 10, \qquad 3 \mapsto 11.$$

All four codewords have length two, so this is a *fixed-length* code, also called a uniform code. Encoding a message means concatenating its codewords. For example,

$$(0,1,3) \longmapsto (00,01,11) \longmapsto 000111.$$

The complete message (1.1) becomes

$$0001111000111010011010010110101001011010100101011001.$$

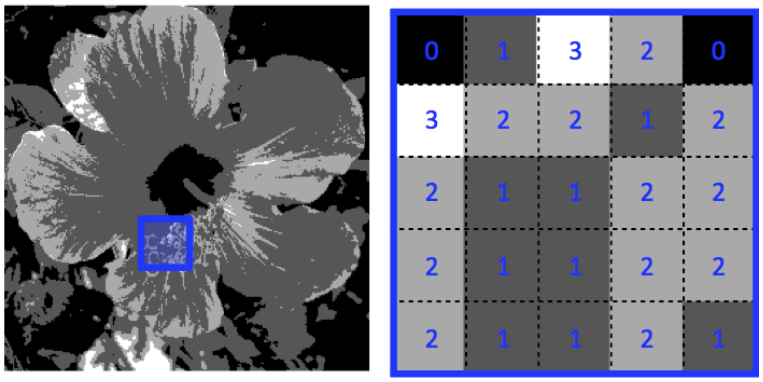


Figure 1.1. A four-level grayscale image and a magnified 5×5 patch.

Its total length is $\overline{L} = 25 \times 2 = 50$ bits. This count concerns the encoded pixel values; a file may also store dimensions, a coding table and other metadata.

For an alphabet of $N \geq 2$ symbols, a fixed-length binary code of length ℓ exists precisely when $2^\ell \geqslant N$: there must be at least as many distinct bit strings as symbols. The smallest possible length is therefore

$$\ell = \lceil \log_2 N \rceil.$$

When $N = 2^\ell$, no bit strings are left unused. For $N = 8$, for example, the codewords have length $\ell = 3$ and are

$$000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111.$$

The base-two logarithm is defined for $x > 0$ by $\log_2 x = \ln x / \ln 2$. It satisfies $\log_2(1/x) = -\log_2 x$; in particular,

$$\log_2(1/4) = -\log_2 4 = -2.$$

The ceiling in the formula for ℓ is essential when N is not a power of two.

1.2.3 Variable-length coding

A code can use fewer bits on average by assigning shorter words to frequent symbols. Consider

$$0 \mapsto 001, \qquad 1 \mapsto 01, \qquad 2 \mapsto 1, \qquad 3 \mapsto 000. \tag{1.2}$$

Now $(0, 1, 3)$ becomes 00101000. The complete message becomes

$$00101000100100011011010111101011110101101,$$

which has 42 bits. The *average length per symbol* is

$$L = \frac{\overline{L}}{25} = \frac{42}{25} = 1.68 \text{ bits}.$$

This code improves on the fixed-length code for this particular message. It need not do so for every message: a sequence consisting mainly of symbols 0 and 3 would require nearly three bits per symbol.

1.2.4 Prefix codes and unambiguous decoding

A code is *uniquely decodable* if two different finite symbol sequences never produce the same bit string. Fixed-length codes with distinct codewords have this property: split the message into blocks of ℓ bits, then read the coding table backwards.

Distinct codewords alone are not sufficient for a variable-length code. For example, the assignment

$$0 \mapsto 0, \quad 1 \mapsto 10, \quad 2 \mapsto 110, \quad 3 \mapsto 101$$

is ambiguous because $1010 = 10\,10 = 101\,0$ represents either $(1, 1)$ or $(3, 0)$.

A sufficient condition for unique decoding is that no codeword be a proper prefix of another. Such a code is called a *prefix code*. The code (1.2) has this property. Its words can be recognized as soon as their last bit is read:

$$\underbrace{001}_{0}\ \underbrace{01}_{1}\ \underbrace{000}_{3}\ \underbrace{1}_{2}\ \cdots .$$

The spaces shown here explain the decoding; they are not transmitted.

The prefix condition is sufficient, but not necessary. The code $A \mapsto 0$, $B \mapsto 01$ is uniquely decodable although 0 prefixes 01. Every 1 must be paired with its preceding 0, so the complete message has only one parsing. A decoder must nevertheless look ahead after reading a 0 to decide whether it represents A or begins B. More general non-prefix codes can require delayed decisions or backtracking; prefix codes avoid this difficulty.

1.2.5 Codes as binary trees

The empty word is the root of a binary tree. Appending a 0 or a 1 selects its left or right child. A word of length ℓ lies at depth ℓ, so the complete tree of depth ℓ contains 2^ℓ leaves and $\ell + 1$ levels, including the root.

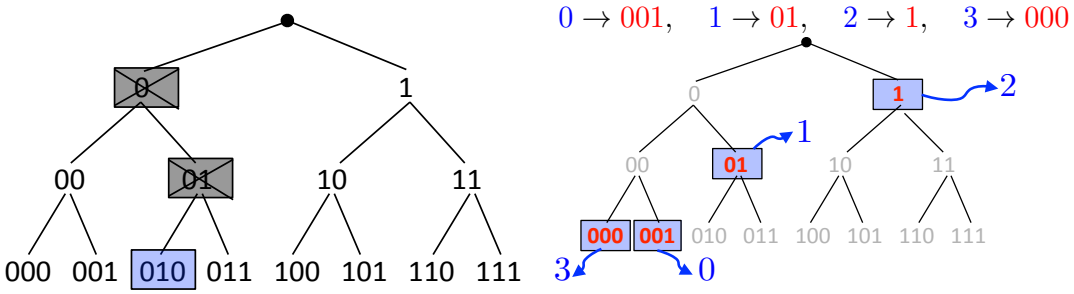


Figure 1.2. The complete binary tree to depth three (left) and the subtree of a prefix code (right).

For a prefix code, no selected codeword is an ancestor of another. The selected nodes are therefore leaves of the tree formed by their paths from the root. To decode, start at the root and follow one edge per bit. On reaching a codeword leaf, output its *symbol* and return to the root. This procedure explains why prefix decoding is immediate and unambiguous.

1.3 Entropy and the Shannon bound

1.3.1 A probabilistic model

The quality of a code depends on the messages to which it is applied. A probabilistic model makes this dependence explicit. Let V be a random symbol taking values in $\{0, \dots, N-1\}$, with probabilities

$$p_v = \mathbb{P}(V = v), \qquad p_v \geqslant 0, \qquad \sum_{v=0}^{N-1} p_v = 1.$$

The simplest model generates a sequence of independent symbols with this same distribution. Independence is an assumption about the source, not a property guaranteed by the coding algorithm.

For an observed message, probabilities can be estimated by *empirical frequencies*. The message (1.1) contains 2 zeros, 9 ones, 12 twos and 2 threes. Its empirical distribution is

$$p = (2/25,\ 9/25,\ 12/25,\ 2/25).$$

Using these frequencies as probabilities is a modeling choice. If the decoder does not already know the chosen model or coding table, the information needed to describe it must also be transmitted.

1.3.2 Entropy

The *entropy* of V, measured in bits, is

$$H(V) = -\sum_{v=0}^{N-1} p_v \log_2 p_v. \tag{1.3}$$

A symbol with probability $p_v > 0$ carries the information $-\log_2 p_v$. Entropy is the average of this quantity. The convention $0 \log_2 0 = 0$ means that impossible symbols contribute nothing; it agrees with $\lim_{x \downarrow 0} x \log_2 x = 0$.

Since $-\log_2 p_v \geqslant 0$, entropy is nonnegative. More precisely,

$$0 \leqslant H(V) \leqslant \log_2 N.$$

The endpoints describe two useful models:

- **No uncertainty.** If $p_1 = 1$, every symbol equals 1 and $H(V) = 0$. The message is a sequence of ones.
- **Maximum uncertainty.** If $p_v = 1/N$ for all v, then $H(V) = \log_2 N$. With four equally likely symbols, this is two bits. A random message from this model need not contain exactly the same number of each symbol.
- **An intermediate case.** For the empirical distribution above,

$$H(V) = -2\frac{2}{25}\log_2\frac{2}{25} - \frac{9}{25}\log_2\frac{9}{25} - \frac{12}{25}\log_2\frac{12}{25} \simeq 1.6219 \text{ bits}.$$

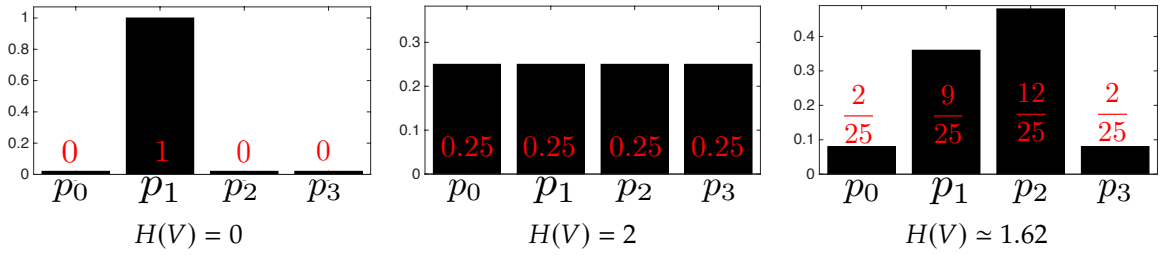


Figure 1.3. A deterministic distribution, the uniform distribution on four symbols, and the empirical distribution of the 25-pixel message.

1.3.3 Expected code length

Let c_v be the codeword for v and ℓ_v its length. The expected number of bits per symbol is

$$L(c) = \mathbb{E}[\ell_V] = \sum_v p_v \ell_v.$$

For the code (1.2), the lengths are $(3, 2, 1, 3)$, hence

$$L(c) = \frac{2 \times 3 + 9 \times 2 + 12 \times 1 + 2 \times 3}{25} = \frac{42}{25} = 1.68.$$

If p is the empirical distribution of a finite message, this formula is also its exact average code length. If p is a source model, it is an expectation.

1.3.4 The entropy lower bound

Every binary prefix code satisfies the Shannon bound [41]

$$H(V) \leqslant L(c). \tag{1.4}$$

Here is a short proof. Let m be the greatest codeword length. A word of length ℓ_v has $2^{m-\ell_v}$ descendants at depth m. Prefix-free words have disjoint sets of descendants, so

$$\sum_v 2^{m-\ell_v} \leqslant 2^m, \qquad Z \stackrel{\text{def.}}{=} \sum_v 2^{-\ell_v} \leqslant 1.$$

This is Kraft's inequality. Define $q_v = 2^{-\ell_v}/Z$. Then $\sum_v q_v = 1$ and $\ell_v = -\log_2 q_v - \log_2 Z$, which gives

$$\begin{aligned} L(c) - H(V) &= \sum_{v : p_v > 0} p_v \log_2 \frac{p_v}{q_v} - \log_2 Z \\ &\geqslant \sum_{v : p_v > 0} p_v \log_2 \frac{p_v}{q_v} \geqslant 0. \end{aligned}$$

The last sum is the Kullback–Leibler divergence. Its nonnegativity follows from $\ln t \leqslant t - 1$: applying this to $t = q_v/p_v$ and summing yields $\sum p_v \ln(q_v/p_v) \leqslant \sum_{p_v>0} q_v - 1 \leqslant 0$.

The bound concerns an *average*; it does not impose $\ell_v \geqslant -\log_2 p_v$ separately for each symbol. The real numbers $-\log_2 p_v$ nevertheless suggest useful target lengths: a *larger* probability gives a smaller target length. For a uniform distribution on 2^ℓ symbols, the fixed-length code attains the bound exactly. For other alphabet sizes, integer codeword lengths can prevent equality.

1.3.5 Huffman coding and blocks

Huffman's algorithm [23] constructs a prefix code with minimum expected length for a given finite distribution. Symbols of probability zero are omitted from the effective alphabet. For a source with at least two positive-probability symbols, its length satisfies the strict upper bound

$$H(V) \leqslant L(c) < H(V) + 1.$$

One way to understand the upper bound is to round each target length up: $\ell_v = \lceil -\log_2 p_v \rceil$. These lengths satisfy Kraft's inequality and can be realized by a prefix code. Their average is less than $H(V) + 1$, and a Huffman code does at least as well.

To reduce the overhead per symbol, encode blocks of n independent symbols. Their entropy is $H(V_1, \ldots, V_n) = nH(V)$, so a suitable block code has expected length L_n satisfying

$$H(V) \leqslant \frac{L_n}{n} < H(V) + \frac{1}{n}.$$

Thus the rate approaches entropy as the block length grows. Arithmetic coding [36] implements related ideas without storing an exponentially large table of block codewords. Finite messages still incur model, termination and numerical-precision overheads.

If a source has only one possible symbol, its entropy is zero. When this symbol and the message length are already known, no payload bits are needed. Requiring a nonempty codeword would instead spend at least one bit per symbol. The convention about framing and the effective alphabet must therefore be stated.

1.3.6 Exploiting dependence between pixels

Neighboring pixels in an image are usually dependent. A symbol-by-symbol model ignores this structure. A useful remedy is to transform the data before coding them.

Index the P pixels by $v_0, \ldots, v_{P-1}$ and define

$$d_i = v_i - v_{i-1}, \qquad 1 \leqslant i \leqslant P - 1.$$

Keep v_0 as well as the differences. The transformation

$$(v_0, \ldots, v_{P-1}) \longmapsto (v_0, d_1, \ldots, d_{P-1})$$

is bijective, since $v_i = v_0 + \sum_{j=1}^{i} d_j$. In a nearly constant region, most differences are zero or small. For four-level pixels, they lie in $\{-3, -2, -1, 0, 1, 2, 3\}$.

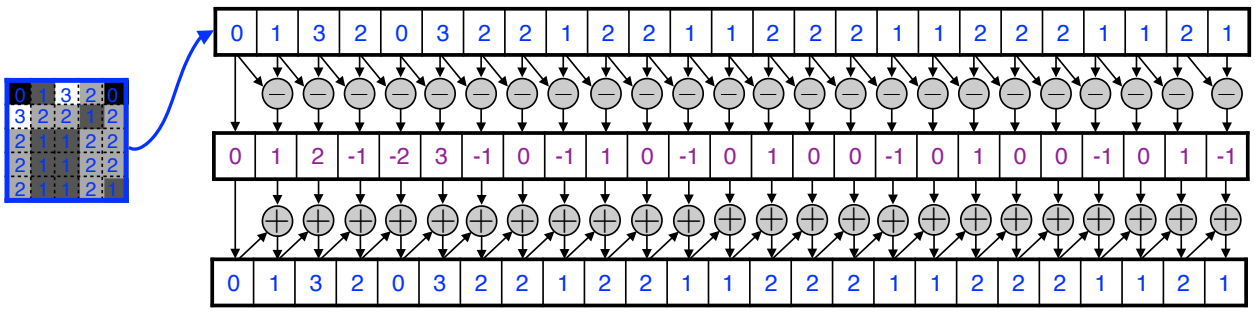


Figure 1.4. Successive differences and reconstruction by cumulative summation, retaining the first pixel.

For the flower example, the difference histogram is more concentrated than the pixel histogram. The empirical entropies and Huffman lengths shown in Figure 1.5 illustrate the resulting gain when values are coded separately. This gain is not guaranteed for every image.

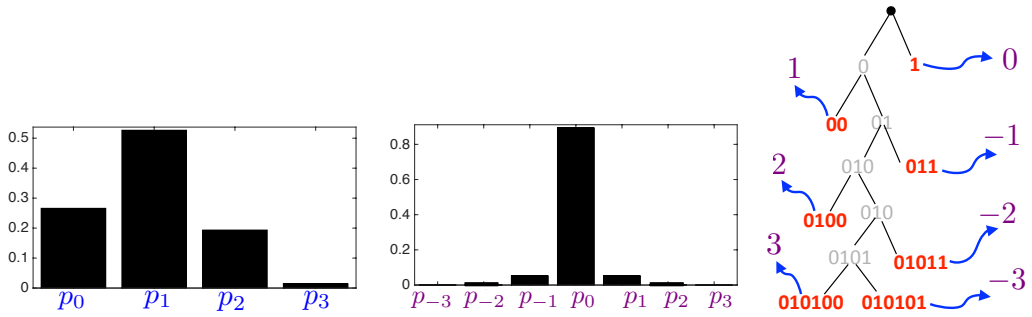


Figure 1.5. Pixel and difference histograms over the entire image, with a prefix-code tree for the differences.

The displayed difference code is

d	-3	-2	-1	0	1	2	3
c_d	010101	01011	011	1	00	0100	010100

and has an average length of approximately 1.16 bits per difference. Using the displayed rounded rates for a 256×256 image gives payload estimates of 16 KiB for the two-bit fixed-length code, 13.36 KiB for direct pixel coding, and 9.28 KiB for difference coding. Here 1 KiB is 1024 bytes, or 8192 bits. The initial pixel and coding-table overhead must be added to the last estimate.

A bijective transformation does *not* reduce the joint entropy of the whole message. It can make separate-symbol models more effective by changing the marginal distributions and the dependence structure. The bound (1.4) itself needs no independence assumption for a single random symbol; independence was used above to obtain the block identity $H(V_1, \ldots, V_n) = nH(V)$. For a dependent source, the joint entropy of the block is the relevant lower bound.

More advanced image codecs combine prediction or transforms with quantization and entropy coding. JPEG 2000, for example, uses wavelets and supports both lossless and lossy modes [29]. The same principle of exploiting dependence also applies to the letters of a text.

Further reading. For further details on source coding, see [12], and [29] for signal and image processing. The original figure programs are available at https://github.com/gpeyre/2016-shannon-theory; related experiments appear in the Numerical Tours [34].

CHAPTER 2

Image processing

A digital image is an array of numbers, so changing an image means computing with those numbers. This chapter develops simple methods for storing images, removing noise, detecting edges and manipulating colors. Each example connects a visible effect with an explicit formula and explains the information that the operation preserves or loses.

2.1 Pixels and grayscale values

A grayscale image with n rows and p columns contains np *pixels*. In the eight-bit representation considered here, each pixel stores an integer between 0 and $255 = 2^8 - 1$. Zero is displayed as black, 255 as white, and the intermediate values as shades of gray. Other bit depths and floating-point representations are also used.

The flower in Figure 2.1 has $256 \times 256 = 65\,536$ pixels. The enlargement shows a 5×5 patch: each displayed shade corresponds to one of the numbers in the array. A photograph may contain millions of pixels, but the same representation applies.

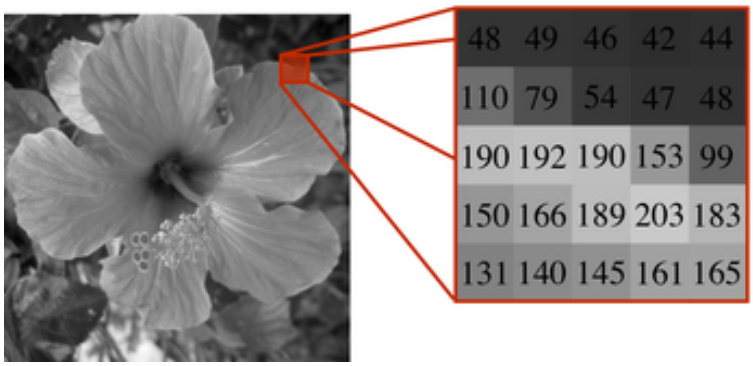


Figure 2.1. An image and a magnified 5×5 patch of pixel values.

2.2 Storing an image

2.2.1 Binary representation and storage units

An integer is represented in binary by decomposing it into powers of two. For example,

$$179 = 2^7 + 2^5 + 2^4 + 2^1 + 2^0 = \underbrace{(10110011)_2}_{\text{binary representation}} .$$

Reading from 2^7 down to 2^0, a bit is one when the corresponding power occurs and zero otherwise. Eight bits suffice for every integer from 0 to 255.

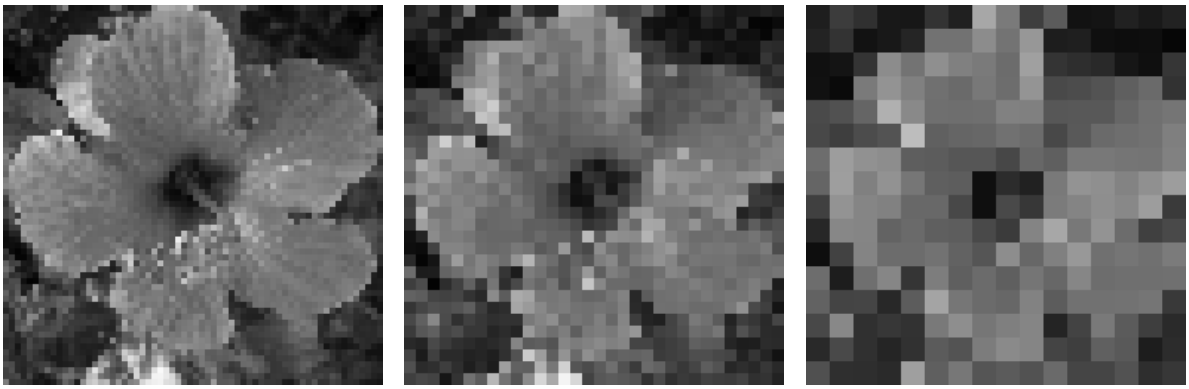
An uncompressed eight-bit grayscale image therefore requires $8np$ bits, or np *bytes*, for its pixel values. The 256×256 flower uses

$$256 \times 256 \times 8 = 524\,288 \text{ bits} = 65\,536 \text{ bytes}.$$

This is 65.536 kB in decimal units, or 64 KiB in binary units: 1 kB is 1000 bytes, whereas 1 KiB is 1024 bytes. For comparison, a 240×240 image would require 57 600 bytes, or 56.25 KiB. File headers and metadata add to these payload sizes.

2.2.2 Subsampling

Reducing the number of pixels reduces storage. Keeping every fourth row and every fourth column divides the pixel count by $4^2 = 16$, when both dimensions are divisible by four. Figure 2.2 shows increasingly coarse grids.



One row/column in 4 | One row/column in 8 | One row/column in 16

Figure 2.2. Subsampling on successively coarser grids.

Discarding rows and columns can lose fine detail and create false patterns, an effect called *aliasing*. A practical resizing method usually smooths the image before reducing its sampling rate. The next chapter explains the mathematical reason for this precaution.

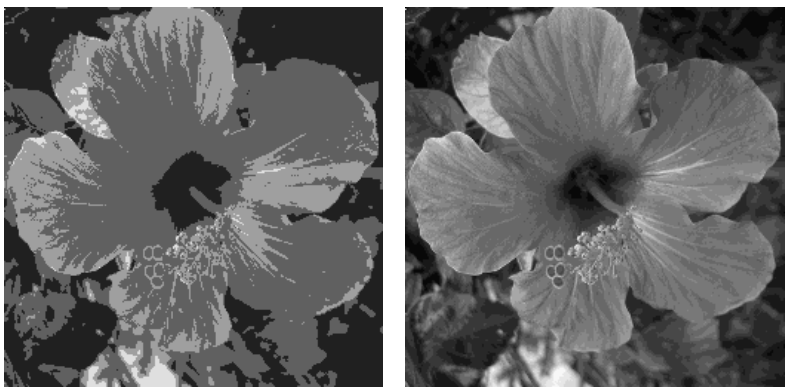
2.2.3 Quantization

Storage can also be reduced by using fewer possible values per pixel. To retain four levels, replace an input value $a \in \{0, \ldots, 255\}$ by

$$q(a) = \left\lfloor \frac{a}{64} \right\rfloor \in \{0, 1, 2, 3\}.$$

Thus 0–63 map to 0, 64–127 to 1, 128–191 to 2, and 192–255 to 3. When displayed on a scale from black to white, these indices correspond respectively to black, dark gray, light gray and white; they can be rendered using the values $85q(a)$.

Four indices require two bits, rather than eight, giving a factor-four saving if the indices are packed into the file. Merely storing the indices in eight-bit containers would not achieve that saving. Figure 2.3 compares four and sixteen levels.



4 gray levels | 16 gray levels

Figure 2.3. Quantization with two different numbers of levels.

Quantization is generally irreversible: many original values share the same index. Lossless compression, by contrast, preserves every input value while exploiting redundancy in its representation. JPEG 2000 uses wavelets and offers both lossless and lossy modes; it is one example of a codec, not a universally best choice

for every image and criterion.[1]

2.3 Removing noise

2.3.1 Local averaging

Noise produces unwanted fluctuations in pixel values. It is particularly visible in low-light photographs, although its distribution depends on the sensor and acquisition process. A simple denoising method replaces each pixel by the mean of a surrounding window.

For the 3×3 neighborhood in Figure 2.4, replace the central value a by

$$\frac{a + b + c + d + e + f + g + h + i}{9}.$$

The displayed values give

$$\frac{190 + 192 + 79 + 54 + 47 + 153 + 203 + 189 + 166}{9} = \frac{1273}{9} \simeq 141.44.$$

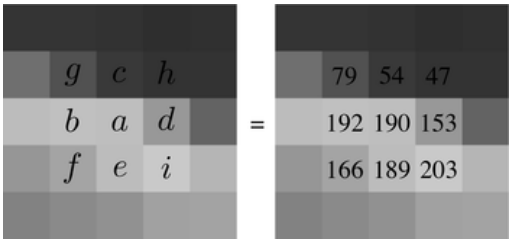
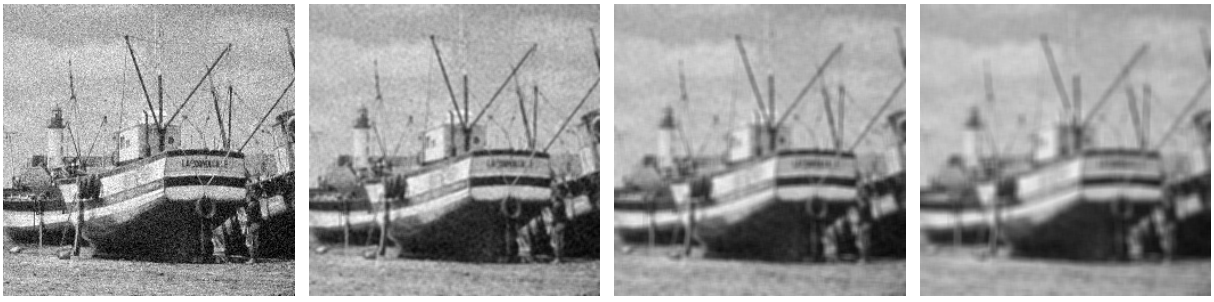


Figure 2.4. A 3×3 neighborhood used for local averaging and median filtering.

All averages must be computed from the same input image. Updating pixels in place and then reusing those updated values defines a different filter. At the image boundary, one must also choose a rule, such as reflecting the image or restricting and renormalizing the window.

Why does averaging help? If a locally constant signal is corrupted by independent, zero-mean noise of variance σ^2, the average of m pixels has noise variance σ^2/m. Near an edge, however, the underlying signal is not constant: averaging mixes distinct regions and introduces blur. A larger window trades more noise reduction for more loss of detail.



Noisy image | 3×3 mean | 5×5 mean | 7×7 mean

Figure 2.5. Local averaging reduces noise but blurs edges as the window grows.

2.3.2 Local median

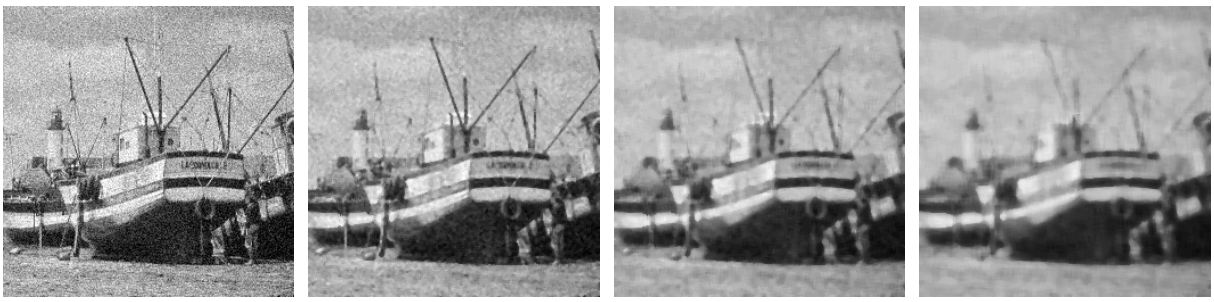
The *median* of an odd number of values is the middle value after sorting. The nine values above, in increasing order, are

$$47, 54, 79, 153, 166, 189, 190, 192, 203,$$

so their median is 166. A median filter replaces each pixel by the median of its input neighborhood.

[1] https://jpeg.org/jpeg2000/

The median is resistant to isolated extreme values and often preserves sharp transitions better than the mean. It is particularly useful for impulse noise, but it is not uniformly better for every noise model or image. Large windows can erase thin objects and small details. Figure 2.6 illustrates this tradeoff.



Noisy image | 3×3 median | 5×5 median | 7×7 median

Figure 2.6. Median filtering with increasing window size.

2.4 Detecting edges

Edges are locations where pixel values change rapidly, for instance at a boundary between a dark object and a bright background. A simple detector compares the neighbors on opposite sides of a pixel.

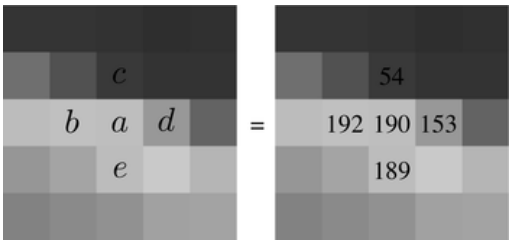


Figure 2.7. The central pixel a and its left, upper, right and lower neighbors b, c, d, e.

With the arrangement of Figure 2.7, define

$$\ell = \sqrt{(b-d)^2 + (c-e)^2}.$$

For unit pixel spacing, $\ell/2$ is the magnitude of the gradient estimated by centered differences. The values in the figure yield

$$\ell = \sqrt{(192-153)^2 + (54-189)^2} = \sqrt{19\,746} \simeq 140.52.$$

We have $\ell = 0$ exactly when $b = d$ and $c = e$. This does not imply that all five values are equal: the detector can miss some fine oscillations. A large value indicates a strong local variation, which may come from an object boundary, texture or noise.

For display, the value is clipped to $\min(\ell, 255)$ and rounded. The minimum is needed because ℓ can exceed 255. Figure 2.8 displays large responses in white. Denoising before differentiation can help because differences also amplify noise.

2.5 Color images

2.5.1 RGB channels and additive color

An RGB image has three channels, representing red, green and blue. They are separate arrays, but are generally *not statistically independent*. In an eight-bit representation, a pixel is a triplet $(r, g, b) \in \{0, \ldots, 255\}^3$ and requires 24 bits before compression.

The values $(255, 0, 0)$, $(0, 255, 0)$ and $(0, 0, 255)$ display red, green and blue respectively. Combining emitted light follows *additive* color mixing: red plus green, $(255, 255, 0)$, gives yellow; red plus blue, $(255, 0, 255)$, gives magenta. White is $(255, 255, 255)$ and black is $(0, 0, 0)$.



Original image Edge response ℓ

Figure 2.8. Edge detection by differences between opposite neighbors.



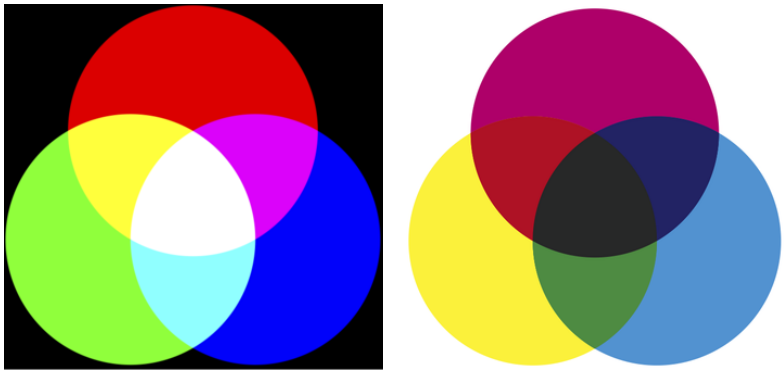
Color image Mean-channel grayscale

Figure 2.9. A color image and a grayscale representation.



Red channel Green channel Blue channel

Figure 2.10. The three RGB channels, displayed in their corresponding colors.



Additive mixing Subtractive mixing

Figure 2.11. Idealized additive and subtractive color mixing.

2.5.2 CMY channels and subtractive color

An idealized complementary representation uses cyan, magenta and yellow:

$$c = 255 - r, \qquad m = 255 - g, \qquad y = 255 - b.$$

For example, blue $(r, g, b) = (0, 0, 255)$ corresponds to $(c, m, y) = (255, 255, 0)$. These variables describe the *subtractive* action of ideal pigments on white light: cyan absorbs red, magenta absorbs green, and yellow absorbs blue.



Cyan channel Magenta channel Yellow channel

Figure 2.12. The complementary CMY channels, shown against white backgrounds.

Real printers use device-dependent color conversions, usually with an additional black channel (CMYK). A screen still emits RGB light, so CMY data must be converted for display. Neither a fixed set of display primaries nor a fixed set of inks reproduces every visible color.

2.6 Changing brightness and contrast

2.6.1 A grayscale intensity

A simple way to obtain a grayscale image is to average the three channels:

$$a = \frac{r + g + b}{3}.$$

This is the intensity used in the elementary examples here. It is not the physical *luminance*, which weights linear-light components according to visual sensitivity. In particular, an arithmetic mean of encoded RGB values is not a calibrated measurement of emitted light.[2]

[2]For the definition of relative luminance in sRGB, see https://www.w3.org/TR/WCAG22/#dfn-relative-luminance.

2.6.2 Pointwise changes in grayscale

Replacing a by $255 - a$ exchanges black and white and produces a negative. To darken an image, use $a \mapsto a^2/255$; to brighten it, use $a \mapsto \sqrt{255a}$. Both maps preserve the interval $[0, 255]$.



Negative | Square: darker | Square root: brighter

Figure 2.13. Three pointwise transformations of grayscale intensity.

The square and square-root maps are inverses as functions on the real interval. After rounding to integer pixel values, they need not undo one another exactly. Their common generalization is the power transformation

$$\widetilde{a} = 255 \left(\frac{a}{255} \right)^{\gamma}, \qquad \gamma > 0,$$

with $\widetilde{a} = 0$ when $a = 0$. For $0 < a < 255$, the image becomes brighter when $\gamma < 1$ and darker when $\gamma > 1$. The cases $\gamma = 1/2$ and $\gamma = 2$ give the square root and square, respectively; $\gamma = 1$ leaves the values unchanged.

2.6.3 A color transformation

Applying unrelated changes to the three channels can alter colors strongly. One elementary alternative changes the mean intensity a while keeping the residual $(r - a, g - a, b - a)$ fixed. Define

$$\begin{aligned} \widetilde{r} &= \operatorname{clip}(r + \widetilde{a} - a), \\ \widetilde{g} &= \operatorname{clip}(g + \widetilde{a} - a), \\ \widetilde{b} &= \operatorname{clip}(b + \widetilde{a} - a), \end{aligned} \qquad \operatorname{clip}(u) = \max(0, \min(255, u)).$$

Without clipping, the new mean is $\widetilde{a}$ and the differences between channels are preserved. Clipping can change both properties, and this construction does not guarantee exact preservation of perceived hue or saturation. Values are rounded only after the computation.



$\gamma = 0.5$ | 0.75 | 1 | 1.5 | 2 | 3

Figure 2.14. Changing a color image through a power transformation of its mean intensity.

2.7 Images as matrices

2.7.1 Transposition and rotation

Write an image as a matrix $A = (a_{i,j}) \in \mathbb{R}^{n \times p}$. Rows are indexed from top to bottom by $1 \leqslant i \leqslant n$ and columns from left to right by $1 \leqslant j \leqslant p$. Its transpose $B = A^{\mathsf{T}} \in \mathbb{R}^{p \times n}$ satisfies

$$b_{j,i} = a_{i,j}.$$

It exchanges the two coordinate directions and reflects the image across the diagonal direction. A clockwise quarter-turn produces another $p \times n$ matrix C given by

$$c_{j,i} = a_{n-i+1,j}, \qquad 1 \leqslant j \leqslant p, \quad 1 \leqslant i \leqslant n.$$

Equivalently, the original pixel (i, j) moves to $(j, n - i + 1)$. These operations permute pixel values and therefore lose no information. Apply the same permutation to all channels of a color image.



A A^{T} Clockwise rotation

Figure 2.15. Transposition and rotation by a quarter-turn.

2.7.2 A crossfade between two images

For images A and B of the same dimensions and channel format, define

$$C_t = (1 - t)A + tB, \qquad 0 \leqslant t \leqslant 1.$$

This formula is applied to each pixel and channel. The endpoints are $C_0 = A$ and $C_1 = B$; intermediate values produce a *crossfade*. Because the weights are nonnegative and sum to one, values stay in $[0, 255]$ before rounding. This blends colors at fixed locations; it does not move objects from one location to another.



$t = 0$ 0.2 0.4 0.6 0.8 1

Figure 2.16. Linear interpolation from image A to image B.

Further reading. Reproducible examples are available in the Numerical Tours [34]. This chapter draws in part on the author's article *Le traitement numérique des images* in *Images des mathématiques*.

CHAPTER 3

Sparsity, inverse problems and compressed sensing

How can a small number of measurements describe a complicated signal? Sparse representations provide a link between compression and the recovery of incomplete or noisy data. This chapter develops that link, explains the role of convex optimization, and states the assumptions under which compressed sensing gives reliable reconstructions.

3.1 From continuous signals to samples

An acquisition device turns a continuous signal $\tilde{f}$ into a finite vector $f = (f_q)_{q=1}^{Q} \in \mathbb{R}^Q$. For sound, these entries represent measurements at successive times. For a grayscale image, they represent light collected over small sensor regions. An idealized pixel measurement is

$$f_q = \int_{c_q} \tilde{f}(s)\,\mathrm{d}s,$$

where c_q is the area of the sensor element. Dividing by its area gives an average intensity; exposure time and calibration introduce further scale factors. Color and multichannel signals can be handled by recording several values at each location.

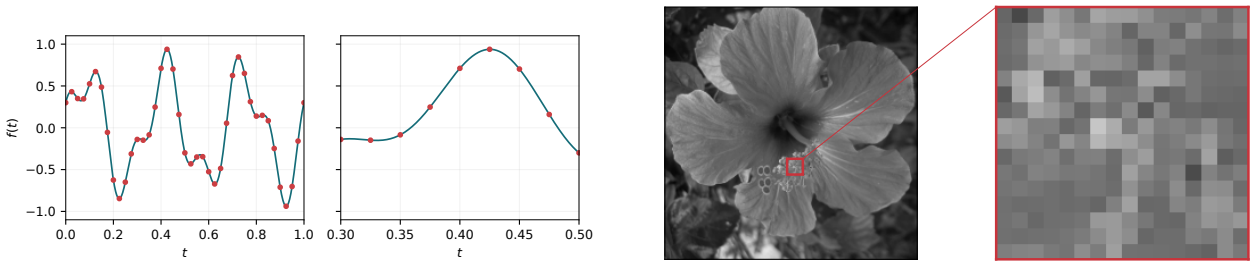


Figure 3.1. Sampling a one-dimensional signal and a two-dimensional image. The dots mark time samples; the red frames identify the image region enlarged on the right.

Sampling preserves information only under suitable assumptions. For example, let $\tilde{f} \in L^2(\mathbb{R})$ have Fourier transform supported in $[-B, B]$, with frequency measured in cycles per unit time. If the sampling interval satisfies $0 < T < 1/(2B)$, the sampling theorem [42] gives

$$\tilde{f}(t) = \sum_{k\in\mathbb{Z}} \tilde{f}(kT)\ \mathrm{sinc}\left(\frac{t}{T} - k\right), \qquad \mathrm{sinc}(u) = \begin{cases} \sin(\pi u)/(\pi u), & u \neq 0, \\ 1, & u = 0. \end{cases}$$

For the continuous representative of this band-limited function, the series converges in L^2 and uniformly. The theorem uses samples over the entire time axis. A finite recording, quantization, and measurement noise introduce additional errors. Frequencies above the acquisition band must be attenuated before sampling to avoid aliasing; increasing numerical precision afterwards cannot undo aliasing.

This is a different question from lossless coding: sampling concerns the representation of a continuous signal by measurements, whereas coding concerns the binary representation of those measurements. Sparsity provides another useful assumption on the signal, and leads to a different acquisition strategy.

3.2 Nonlinear approximation and compression

3.2.1 Sparse representations

A dictionary $\Psi = (\psi_n)_{n=1}^N$ consists of vectors $\psi_n \in \mathbb{R}^Q$. We seek an approximation

$$f \approx \Psi x = \sum_{n=1}^{N} x_n \psi_n$$

using few nonzero coefficients. Sinusoids, local cosine functions, and wavelets are familiar examples of atoms. Transform coding uses such representations to concentrate much of a signal's energy in a small number of coefficients.

Given an integer budget $0 \le M \le N$, a best M-term approximation solves

$$x^\star \in \arg\min_{x \in \mathbb{R}^N,\ \|x\|_0 \le M} \|f - \Psi x\|_2^2, \qquad \|x\|_0 = \#\{n : x_n \neq 0\}. \tag{3.1}$$

Here $\|v\|_2^2 = \sum_n |v_n|^2$. The notation $\|x\|_0$ is conventional: this count is not a norm, since multiplying a nonzero vector by a nonzero scalar leaves the count unchanged. For a general dictionary, sparse approximation is NP-hard [33]. Exhaustively testing the possible supports is usually prohibitive; this worst-case difficulty does not rule out efficient algorithms for special dictionaries or particular data.

3.2.2 The orthonormal case

Suppose $N = Q$ and Ψ is an orthonormal basis, so that $\langle \psi_n, \psi_m \rangle = \delta_{nm}$. With $a_n = \langle f, \psi_n \rangle$, Parseval's identity gives

$$f = \sum_{n=1}^{N} a_n \psi_n, \qquad \|f - \Psi x\|_2^2 = \sum_{n=1}^{N} |a_n - x_n|^2. \tag{3.2}$$

Once a support is chosen, the best coefficients are the corresponding a_n. To minimize the discarded energy, keep the M largest magnitudes. More precisely, choose an ordering $|a_{n_1}| \ge \cdots \ge |a_{n_N}|$. Then

$$x_n^\star = \begin{cases} a_n, & n \in \{n_1, \dots, n_M\}, \\ 0, & \text{otherwise.} \end{cases} \tag{3.3}$$

Ties at the threshold can produce several minimizers. If fewer than M coefficients are nonzero, the approximation uses fewer than M terms.



f $M = N/4$ $M = N/8$ $M = N/16$

Figure 3.2. Best M-term wavelet approximations of a flower image with $N = 256^2$ pixels. The orthonormal transform uses Daubechies wavelets with four vanishing moments, five levels, and periodic boundary conditions. Display intensities are clipped to $[0, 1]$.

The reconstruction $x \mapsto \Psi x$ is linear, but selecting the largest coefficients depends on the data. The approximation $f \mapsto \Psi x^\star$ is therefore nonlinear. For many piecewise smooth signals, wavelets give small errors with relatively few coefficients; the regularity of the signal controls the decay of this error [29, Chapter 9]. Textures and fine oscillations can require substantially more terms.

The fraction M/N counts retained coefficients; it is not a bit rate. A complete compressor must also quantize the retained values, describe their locations, and encode the resulting symbols. Entropy coding, introduced in the chapter on information theory, completes this step. Wavelet transforms also admit fast algorithms whose cost is proportional to the number of samples [29, Chapter 7].

3.3 Inverse problems and regularization

3.3.1 The observation model

An inverse problem asks us to recover an unknown signal $f \in \mathbb{R}^Q$ from measurements

$$y = \Phi f + w \in \mathbb{R}^P. \tag{3.4}$$

The known linear operator $\Phi \in \mathbb{R}^{P \times Q}$ describes acquisition, and w represents unknown errors. Usually one applies Φ by an algorithm rather than storing this potentially enormous matrix.



f Φf: blur Φf: mask

Figure 3.3. Two noiseless observation operators applied to the same flower image: Gaussian blurring and a mask retaining about 10% of the pixels. Missing entries are displayed in black.

Several standard problems fit this model:

- Denoising: $\Phi = \mathrm{Id}$ and $P = Q$.
- Deblurring: $\Phi f = \varphi \star f$, with a filter φ describing the blur and a specified boundary convention.
- Missing data: Φ selects observed coordinates. Equivalently, one may keep the full array and use the diagonal mask $\Phi = \mathrm{diag}(\mu_q)$, where $\mu_q \in \{0, 1\}$.
- Tomography: Φ models line integrals of the object, as in a discretized Radon transform.

If Φ has a nontrivial kernel, the measurements alone do not determine f. Even when a square Φ is invertible, its small singular values can amplify errors in $\Phi^{-1}y = f + \Phi^{-1}w$. Regularization introduces additional information to select a useful, stable reconstruction.

3.3.2 Sparse regularization

Assuming $f \approx \Psi x$ with few active coefficients leads to

$$x^\star \in \arg\min_{\|x\|_0 \le M} \|y - \Phi\Psi x\|_2^2, \qquad f^\star = \Psi x^\star. \tag{3.5}$$

For denoising in an orthonormal basis, this is hard thresholding of the coefficients of the *observed signal* y. It gives the best sparse fit to y for the chosen budget; it does not by itself guarantee the smallest error relative to the unknown clean signal f. For a general acquisition operator, $\Phi\Psi$ is not orthogonal, and the simple thresholding formula no longer solves the problem.

Greedy algorithms and convex relaxation provide two ways to approach this difficulty [30, 45]. To motivate convex relaxation, consider

$$\|x\|_\alpha^\alpha = \sum_{n=1}^{N} |x_n|^\alpha, \qquad \alpha > 0.$$

For each fixed vector x, this quantity tends to $\|x\|_0$ as $\alpha \downarrow 0$. The sets $B_\alpha = \{x : \sum_n |x_n|^\alpha \le 1\}$ lie in $[-1, 1]^N$ and converge, in Hausdorff distance, to

$$B_0 = \{x \in [-1, 1]^N : \|x\|_0 \le 1\}.$$

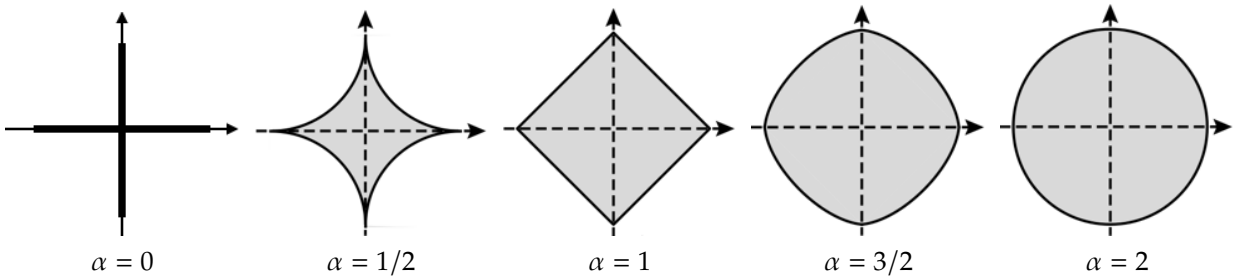


Figure 3.4. The sets B_α in two dimensions. The first panel shows the bounded limiting set B_0.

This limiting set consists of the coordinate-axis segments, including the origin. The set $\{x : \|x\|_0 \le 1\}$ without the box restriction instead contains the entire, unbounded coordinate axes.

Within this family, $\|\cdot\|_\alpha$ is a norm and B_α is convex precisely when $\alpha \ge 1$. The choice $\alpha = 1$ balances convexity with a geometry that favors zero coefficients. This leads to the constrained Lasso problem

$$x^\star \in \arg\min_{\|x\|_1 \le \tau} \frac{1}{2}\|Ax - y\|_2^2, \qquad A = \Phi\Psi. \tag{3.6}$$

Increasing τ allows a closer fit to the measurements, but can also fit their errors. The parameter must be chosen with the noise level, operator, and signal model in mind. Sending τ to infinity removes the regularization and generally leaves many possible reconstructions when $P < N$.

Two closely related formulations are useful:

$$\min_x \ \frac{1}{2}\|Ax - y\|_2^2 + \lambda\|x\|_1, \qquad \lambda > 0, \tag{3.7}$$

$$\min_x \|x\|_1 \quad \text{subject to } \|Ax - y\|_2 \le \varepsilon, \qquad \varepsilon \ge 0. \tag{3.8}$$

The penalized formulation trades data fit against sparsity. The second formulation, basis pursuit denoising, is convenient when an error bound ε is known. For $\varepsilon = 0$, it becomes basis pursuit, $\min_{Ax=y}\|x\|_1$. These formulations are connected through suitable Lagrange multipliers; their parameters are not interchangeable numerical values. Early developments include seismic inversion, basis pursuit, and the Lasso [10, 40, 44].

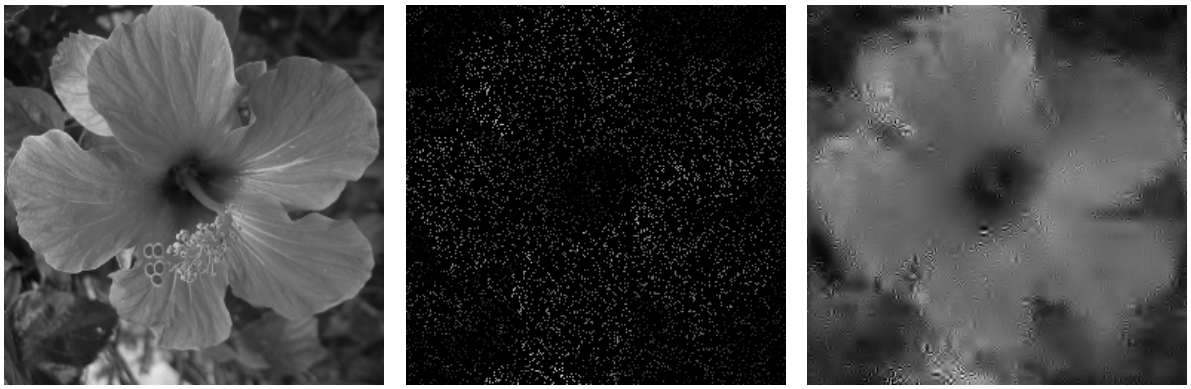


Figure 3.5. Reconstruction from 6,554 observed pixels out of 256^2 (about 10%). The penalized problem (3.7) is solved numerically with $\lambda = 0.005$, intensities in $[0, 1]$, and the same orthonormal wavelets as in Figure 3.2. The remaining texture errors illustrate the limits of the model.

Convexity makes global optimization possible, but the large dimension and the nondifferentiability of $\|x\|_1$ still require suitable algorithms. Proximal methods exploit the explicit soft-thresholding operation associated with this penalty [11]. A numerical solver reaches a prescribed accuracy, rather than an exact real-valued solution after a guaranteed small number of operations.

3.3.3 Why the geometry matters

In two dimensions, an ℓ^1 ball has corners on the coordinate axes. A level set of the data-fitting term can first meet the ball at one of these corners, producing a zero coefficient. The round ℓ^2 ball typically gives

two nonzero coefficients in the same configuration (Figure 3.6). This is an explanation of a tendency, not a guarantee for every operator or every datum: a contact along a face can give several minimizers, and an ℓ^1 solution need not be sparse enough to recover the truth.

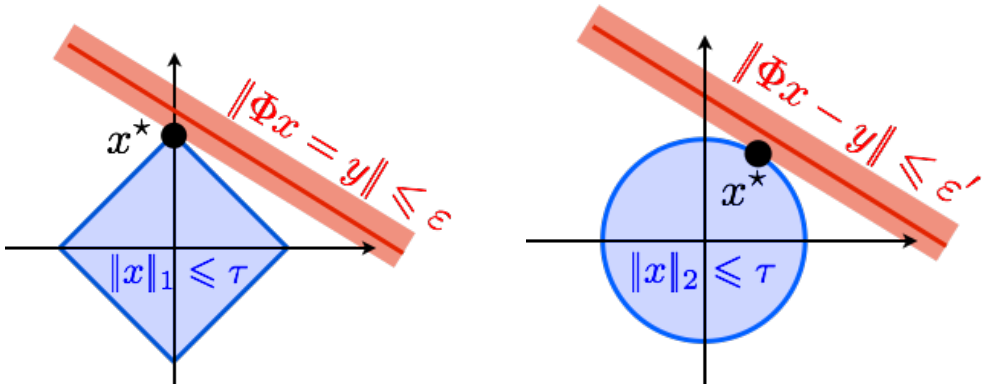


Figure 3.6. Constrained data fitting with an ℓ^1 ball (left) and an ℓ^2 ball (right). Starting from $\varepsilon = 0$, enlarge the strip $\{x : \|Ax - y\|_2 \leq \varepsilon\}$ until it first meets the constraint ball. The contact point minimizes the residual over that ball.

Recovery depends on how the measurement operator acts on sparse vectors. Increasing the ambient dimension alone does not improve it. Precise conditions are available for many random and structured operators; convolution and Fourier acquisition require assumptions adapted to their geometry [1, 8, 9, 18].

3.4 Compressed sensing

3.4.1 Measuring combinations of pixels

Conventional transform coding first acquires Q samples and then keeps a much smaller representation. Compressed sensing asks whether we can instead acquire $P \ll Q$ carefully chosen linear measurements and reconstruct a signal that is sparse, or approximately sparse, in a known basis.

The single-pixel camera developed at Rice University illustrates this idea [17]. A grid of Q micromirrors directs selected portions of the light towards one detector; other mirrors direct light away from it. For each pattern p, the detector measures the sum

$$y_p = \sum_{q=1}^{Q} \Phi_{p,q} \int_{c_q} \tilde{f}(s)\,\mathrm{d}s + w_p = \sum_{q=1}^{Q} \Phi_{p,q} f_q + w_p, \qquad \Phi_{p,q} \in \{0,1\}.$$

The mirrors do not record individual pixels. Changing their configuration produces the vector $y \in \mathbb{R}^P$ directly. The scene must remain sufficiently stable during these measurements. Complementary patterns, or a calibrated measurement of total light, can convert 0/1 measurements into centered signed measurements.

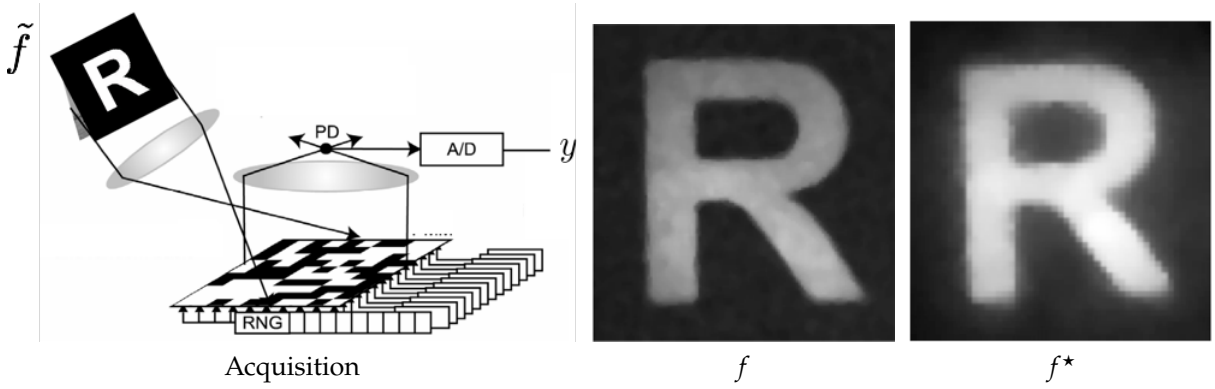


Figure 3.7. Single-pixel acquisition and the original reconstruction example, reproduced from the material accompanying [17]. The stated measurement reduction factor is $Q/P = 6$, so that $P/Q \approx 1/6$.

3.4.2 A recovery guarantee

Here is a standard form of the compressed-sensing guarantee [7, 16]. Suppose Ψ is an orthonormal basis of $\mathbb{R}^N$, $f = \Psi x_0$, and $A = \Phi\Psi$ has independent Gaussian entries with mean zero and variance $1/P$. Equivalently, Φ may be chosen with that Gaussian distribution, since it is invariant under multiplication by a fixed orthogonal matrix.

There are universal positive constants C, c, C_0, C_1 such that, for $1 \le M \le N$ and

$$P \ge CM \log(eN/M), \tag{3.9}$$

with probability at least $1 - 2e^{-cP}$ over the choice of A, the following bound holds simultaneously for all x_0 and all errors with $\|w\|_2 \le \varepsilon$. Any solution $x^\star$ of (3.8), with $y = Ax_0 + w$, satisfies

$$\|x^\star - x_0\|_2 \le C_0 \frac{\sigma_M(x_0)_1}{\sqrt{M}} + C_1\varepsilon, \qquad \sigma_M(x_0)_1 = \min_{\|z\|_0 \le M} \|x_0 - z\|_1. \tag{3.10}$$

The logarithm in (3.9) is natural; changing its base only changes the constant. The interesting compressed regime is one in which the required P is smaller than N.

The quantity $\sigma_M(x_0)_1$ is the sum of the magnitudes of the discarded coefficients after keeping the M largest. If x_0 is exactly M-sparse and the noiseless problem uses $\varepsilon = 0$ (hence $w = 0$), both terms in (3.10) vanish: basis pursuit recovers x_0 exactly. For approximately sparse signals and noisy measurements, the two terms separate the approximation error from the measurement error. Orthonormality gives the same Euclidean error for the reconstructed signal, $\|\Psi x^\star - f\|_2 = \|x^\star - x_0\|_2$.

One route to this theorem is the restricted isometry property: A approximately preserves Euclidean lengths of vectors supported on sufficiently small sets. Random matrices satisfy this property when the number of measurements has the order in (3.9). Appropriate centered subgaussian designs also admit guarantees, with constants depending on their distribution. A raw $0/1$ sensing matrix requires the centering and normalization to be accounted for; it is not the Gaussian model stated above.

The theorem concerns a number of real measurements, not a number of bits. Their precision, quantization, and storage must still be considered. The sensing matrix must be known to the reconstruction algorithm, for instance through a shared random seed. Keeping that matrix secret is not, by itself, a cryptographic security guarantee. Likewise, savings in acquisition time or energy depend on the physical apparatus and the reconstruction cost.

Structured acquisition, including magnetic resonance imaging, imposes additional constraints: one cannot generally choose arbitrary independent Gaussian measurements. Sampling trajectories and sparsity models must therefore be analyzed together [1, 9]. The guiding principle remains to design acquisition and reconstruction jointly, using a model that reflects the data.

Further reading. The interplay between approximation, inverse problems, and optimization extends beyond sparsity. Other penalties encode smoothness, low rank, or geometric structure. The review [45] develops these ideas, and the Numerical Tours [34] provide computational examples.

CHAPTER 4

Optimal transport

Comparing two distributions requires a notion of the cost of moving mass between locations. Optimal transport turns that idea into an optimization problem. Starting with a small assignment problem, we introduce transport plans, explain when they coincide with maps, and use transport distances to compare colors, shapes, and texts.

4.1 Monge's problem

In 1781, Gaspard Monge formulated the problem of moving earth from one place to another at minimum cost [32]. His cost was the distance traveled by each unit of mass. Other applications suggest other costs, such as travel time or the squared Euclidean distance.

4.1.1 An assignment problem

Consider six bakeries and six cafés in Paris. Each bakery supplies one batch of croissants, and each café requires one batch. In the assignment model, each batch goes to exactly one café, and each café receives exactly one batch. Both requirements matter: merely giving every bakery a destination would allow several bakeries to choose the same café.

Let C_{ij} be the travel time from bakery i to café j. The example in Figure 4.1 has $C_{34} = 10$. An admissible assignment is a permutation σ of $\{1, \dots, 6\}$. For instance,

$$(\sigma(1), \dots, \sigma(6)) = (5, 2, 6, 1, 3, 4). \tag{4.1}$$

Its total cost is

$$\text{Cost}(\sigma) = \sum_{i=1}^{6} C_{i,\sigma(i)} = 10 + 7 + 15 + 10 + 14 + 9 = 65. \tag{4.2}$$

This adds the six journey times. It does not model a single vehicle making a tour, nor the completion time of parallel deliveries.

c_{ij}	y_1	y_2	y_3	y_4	y_5	y_6
x_1	12	10	31	27	10	30
x_2	22	7	25	15	11	14
x_3	19	7	19	10	15	15
x_4	10	6	21	19	14	24
x_5	15	23	14	24	31	34
x_6	35	26	16	9	34	15

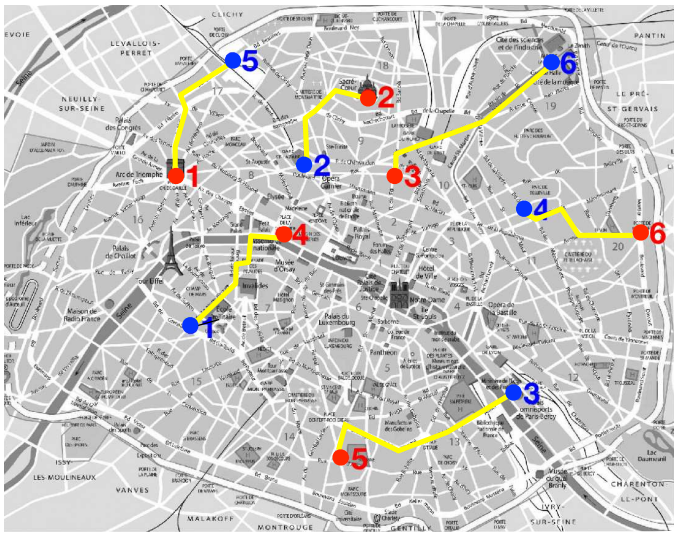


Figure 4.1. Cost matrix and an assignment of the six bakeries (red) to the six cafés (blue). The highlighted entries give a total journey time of 65 minutes.

For n equal batches on each side, the discrete Monge problem is

$$\min_{\sigma \in \Sigma_n} \sum_{i=1}^{n} C_{i,\sigma(i)}, \tag{4.3}$$

where Σ_n is the set of permutations. In our example, the optimal value is 64, achieved by $(5, 6, 2, 1, 3, 4)$. Exhaustively checking the $6! = 720$ possibilities verifies this claim. There are $n!$ assignments in general, and $70! \approx 1.198 \times 10^{100}$, so enumeration quickly becomes impractical.

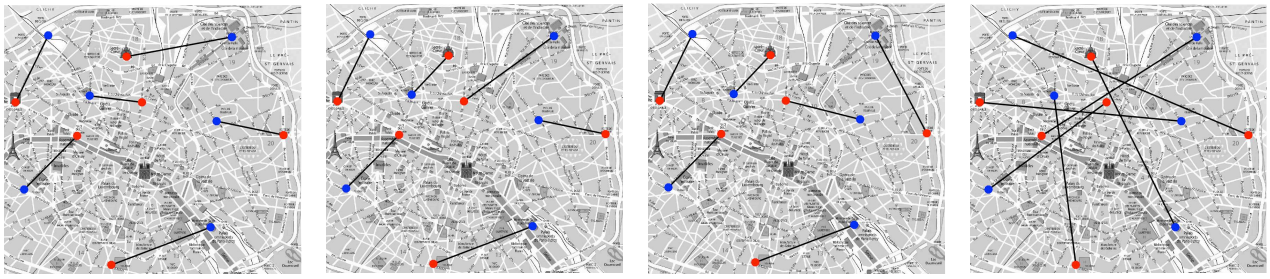


Figure 4.2. Four assignments with costs 64, 65, 66, and 152, from left to right.

4.1.2 Sorting in one dimension

Suppose the source locations $x_1 \le \cdots \le x_n$ and target locations $y_1 \le \cdots \le y_n$ lie on a line, with cost $C_{ij} = |x_i - y_j|$. Matching the points in sorted order is optimal. The key inequality is

$$|x - y| + |x' - y'| \le |x - y'| + |x' - y| \quad \text{when } x \le x' \text{ and } y \le y'.$$

One can verify it by ordering the four points on the line. Whenever a permutation contains an inversion, swapping the two assigned targets does not increase the cost. Removing inversions successively produces the sorted matching. Equality can occur, so this optimal assignment need not be unique.

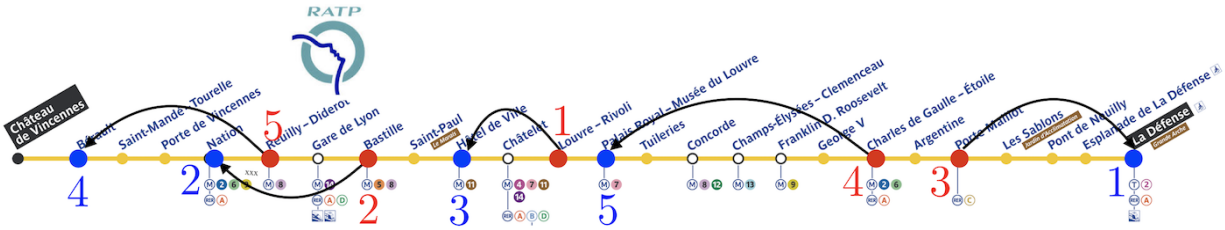


Figure 4.3. Sorted matching along a metro line, considered as a one-dimensional axis. For the indices shown, $\sigma : (1, 2, 3, 4, 5) \mapsto (3, 2, 1, 5, 4)$.

The computational cost is therefore that of sorting the two lists. Insertion sort uses at most $n(n-1)/2$ comparisons per list, which is 2,415 for $n = 70$. Merge sort reduces this to order $n \log n$. These are comparison counts for sorting, rather than estimates of every operation needed by a transport application.

In two dimensions there is no single ordering that solves the general problem. For the Euclidean distance cost, however, two segments in an optimal matching cannot cross transversely in their interiors. If segments $[x, y]$ and $[x', y']$ meet at z, the triangle inequality gives

$$|x - y'| + |x' - y| \le |x - z| + |z - y'| + |x' - z| + |z - y| = |x - y| + |x' - y'|.$$

For a transverse crossing the inequality is strict, so exchanging destinations decreases the cost. Collinear overlaps or shared endpoints are degenerate cases and need not give a strict improvement. This observation does not suffice to compute an optimum: many noncrossing assignments can have different costs.

4.2 Kantorovich's relaxation

Kantorovich's formulation allows mass to split between destinations [24]. It replaces discrete choices by nonnegative quantities subject to linear conservation constraints, making optimal transport a linear programming problem.

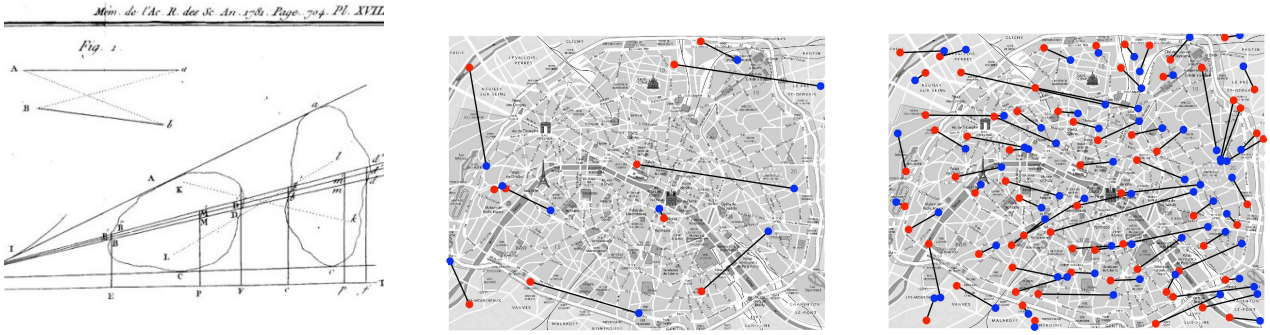


Figure 4.4. An illustration from Monge's original article [32] and two optimal Euclidean matchings in the plane.

4.2.1 From permutations to matrices

Represent a permutation by $P_{ij} = 1$ when $j = \sigma(i)$ and $P_{ij} = 0$ otherwise. For example, the permutations $(1,2,3)$, $(3,2,1)$, and $(2,1,3)$ correspond to

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Each row and each column sums to one. If $\mathcal{P}_n$ denotes this set of permutation matrices, then (4.3) is equivalent to

$$\min_{P \in \mathcal{P}_n} \langle C, P \rangle, \qquad \langle C, P \rangle = \sum_{i,j} C_{ij} P_{ij}. \tag{4.4}$$

Relaxing the binary constraint gives the doubly stochastic matrices

$$\mathcal{B}_n = \left\{ P \in \mathbb{R}_+^{n \times n} : \sum_j P_{ij} = 1, \ \sum_i P_{ij} = 1 \right\}. \tag{4.5}$$

Nonnegativity and the row sums already imply $P_{ij} \le 1$. The relaxed problem is

$$\min_{P \in \mathcal{B}_n} \langle C, P \rangle. \tag{4.6}$$

The set $\mathcal{B}_n$ is compact and convex: for $P, Q \in \mathcal{B}_n$ and $0 \le t \le 1$, the matrix $(1-t)P + tQ$ is still feasible. A minimizer exists because the objective is continuous.

The development of linear programming, including Dantzig's simplex method [14], made these problems computationally accessible. Specialized assignment algorithms, such as implementations of the Hungarian method, solve the dense $n \times n$ assignment problem in $O(n^3)$ operations. This bound should not be attributed to the general simplex method, whose worst-case running time is not polynomial.

4.2.2 Why the relaxation is exact for equal masses

The Birkhoff–von Neumann theorem states that every doubly stochastic matrix is a convex combination of permutation matrices [4, 47]:

$$P = \sum_{k=1}^{K} \lambda_k P^{(k)}, \qquad \lambda_k \ge 0, \quad \sum_k \lambda_k = 1.$$

Consequently, $\langle C, P \rangle = \sum_k \lambda_k \langle C, P^{(k)} \rangle$ cannot be smaller than the least permutation cost. Since every permutation matrix is feasible for the relaxation, we obtain

$$\min_{P \in \mathcal{B}_n} \langle C, P \rangle = \min_{P \in \mathcal{P}_n} \langle C, P \rangle. \tag{4.7}$$

Thus at least one optimal plan is a permutation. It does not follow that every optimal plan is a permutation: the average of two distinct optimal permutations is also optimal. If all costs vanish, every doubly stochastic matrix is optimal.

A useful way to certify an optimal value is to find numbers u_i, v_j satisfying $u_i + v_j \le C_{ij}$. Every $P \in \mathcal{B}_n$ then satisfies

$$\langle C, P \rangle \ge \sum_{i,j} (u_i + v_j) P_{ij} = \sum_i u_i + \sum_j v_j.$$

For the Paris example, $u = (2, 3, 3, 0, 5, 2)$ and $v = (10, 4, 9, 7, 8, 11)$ satisfy all 36 inequalities and their entries sum to 64. Together with an assignment of cost 64, they prove optimality without listing all permutations.

4.2.3 Different numbers of points and unequal masses

Now let n bakeries supply amounts $a_i \ge 0$, and let m cafés require amounts $b_j \ge 0$, with common total mass

$$\sum_{i=1}^{n} a_i = \sum_{j=1}^{m} b_j = s.$$

A transport plan belongs to

$$\mathcal{B}(a, b) = \left\{ P \in \mathbb{R}_+^{n \times m} : \sum_j P_{ij} = a_i,\ \sum_i P_{ij} = b_j \right\}.$$

When $s > 0$, the plan $P_{ij} = a_i b_j / s$ proves feasibility. When $s = 0$, the only plan is zero. The transport problem is

$$\min_{P \in \mathcal{B}(a,b)} \sum_{i=1}^{n} \sum_{j=1}^{m} C_{ij} P_{ij}. \tag{4.8}$$

For finite costs, compactness again ensures the existence of a minimizer. An entry P_{ij} is the amount shipped from i to j, and C_{ij} is its cost per unit of mass.

	3	23	18	26	16	14
24	1	7	0	0	16	0
9	0	0	0	0	0	9
3	0	0	0	0	0	3
16	0	16	0	0	0	0
21	2	0	18	1	0	0
27	0	0	0	25	0	2

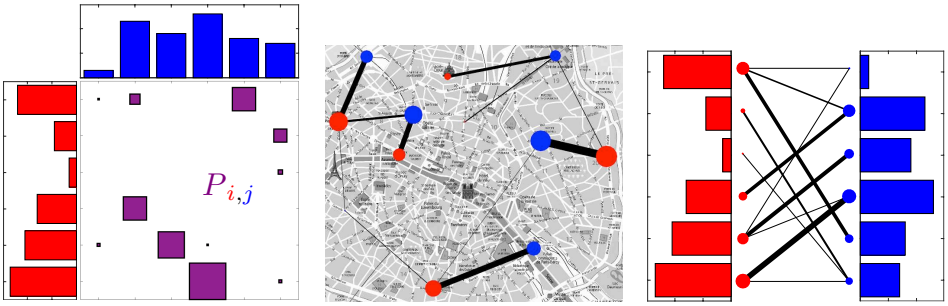


Figure 4.5. Four views of the same coupling: its matrix entries, a display of their magnitudes by squares, weighted segments in space, and a bipartite graph. Row and column sums give the prescribed masses a and b.

Mass splitting can now be necessary. One bakery with mass 2 and two cafés requiring mass 1 each have the unique feasible plan $P = (1\ 1)$. A map that sends the entire source mass to a single target cannot realize these marginals. This explains why a transport plan is more general than a transport map. In the equal-mass case $a_i = b_j = 1$, we recover $\mathcal{B}_n$; for probability masses $a_i = b_j = 1/n$, the corresponding plans are P/n.

4.3 Measures and transport distances

The same ideas apply to probability measures, including discrete distributions $\mu = \sum_i a_i \delta_{x_i}$ and $\nu = \sum_j b_j \delta_{y_j}$. A coupling $\pi \in \Pi(\mu, \nu)$ is a probability measure on pairs (x, y) with marginals μ and ν. A measurable map T induces a coupling $(\mathrm{Id}, T)_{\#}\mu$ when $T_{\#}\mu = \nu$; the notation means that applying T to a random variable of law μ gives one of law ν.

For probability measures on $\mathbb{R}^d$ with finite second moments, the quadratic transport distance is

$$W_2(\mu, \nu)^2 = \min_{\pi \in \Pi(\mu,\nu)} \int |x - y|^2 \, \mathrm{d}\pi(x, y). \tag{4.9}$$

In the discrete case, this is (4.8) with $C_{ij} = |x_i - y_j|^2$. The distance depends on both masses and locations: equal weight vectors at different locations need not describe the same measure. The square root W_2 is a metric,

and $W_2(\mu, \nu) = 0$ exactly when $\mu = \nu$. Its square generally fails the triangle inequality; for point masses at 0, 1, and 2, the relevant squared distances are 1, 1, and 4.

Brenier's theorem gives a precise condition under which the optimal coupling is a map [5]. If the source μ is absolutely continuous with respect to Lebesgue measure and both measures have finite second moments, the quadratic problem has a unique optimal coupling. It is induced by a map $T = \nabla\varphi$, where φ is convex, and T is unique μ-almost everywhere. Absolute continuity and the choice of cost matter; one cannot assert this conclusion for arbitrary measures and arbitrary costs.

4.4 Applications

4.4.1 Transferring a color palette

For two images with the same number n of pixels, let $x_i, y_j \in \mathbb{R}^3$ be their RGB colors. An assignment minimizing $\sum_i |x_i - y_{\sigma(i)}|^2$ gives an output whose color at source position i is $y_{\sigma(i)}$. Every target color is used once, so the output has exactly the target's empirical color distribution. The optimization limits changes in color, but contains no spatial regularity term; preservation of edges and textures is therefore not guaranteed.



Figure 4.6. Color-palette transfer. Columns: source, target, and recolored source. Top: colors on the image grid; bottom: the corresponding point clouds in RGB space. A permutation of the target colors gives the same color cloud in the last two columns.

This distinction separates color transport from moving pixels in the image plane. Here the locations being transported are points in color space; the display grid stays fixed. With different image sizes or weighted color clusters, one uses a coupling and a specified rule to convert that coupling into output colors. A barycentric average of coupled colors need not preserve the target histogram exactly.

4.4.2 Barycenters of shapes

The Euclidean weighted mean of three points minimizes $\alpha|x - r|^2 + \beta|y - r|^2 + \gamma|z - r|^2$, where $\alpha, \beta, \gamma \geq 0$ and $\alpha + \beta + \gamma = 1$. Its value is $r = \alpha x + \beta y + \gamma z$. Replacing squared Euclidean distances by squared transport distances defines a Wasserstein barycenter [2]:

$$\min_{\rho \in \mathcal{P}_2(\mathbb{R}^d)} \alpha W_2(\mu, \rho)^2 + \beta W_2(\nu, \rho)^2 + \gamma W_2(\eta, \rho)^2. \tag{4.10}$$

Here $\mathcal{P}_2$ is the set of probability measures with finite second moment. In a grid-based computation, one instead restricts ρ to probability weights on that grid. A shape can be represented by a uniform density inside it, normalized to total mass one. Changing the three weights produces a family of interpolated distributions (Figure 4.7). These distributions need not themselves be uniform inside a sharp shape, and uniqueness requires additional assumptions.

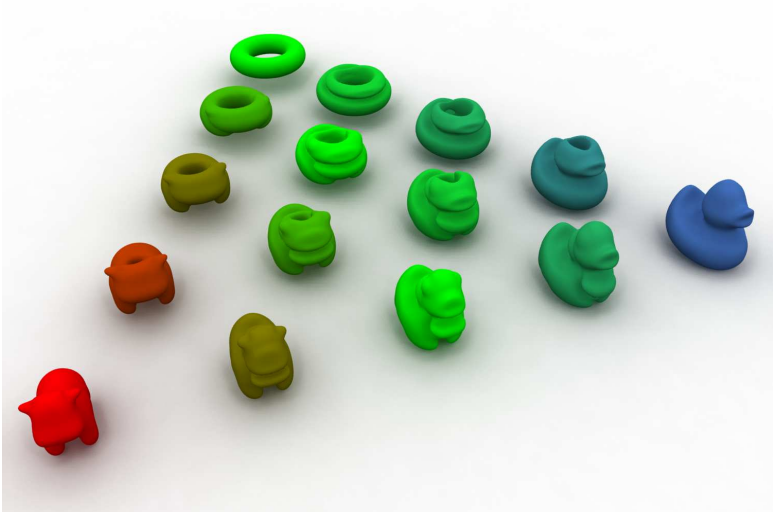


Figure 4.7. Numerical barycentric interpolation of three-dimensional shapes represented by probability distributions. The three reference shapes occupy the vertices of the triangle; interior points correspond to different weights in (4.10).

4.4.3 Comparing texts

Normalize word counts in a text to form a probability distribution. Transport can compare two such distributions using a cost that describes the dissimilarity between words, for example distances between word embeddings. The choice of this ground cost determines which substitutions are considered inexpensive; it may also be learned from a task [21]. Word histograms discard word order, so they cannot by themselves capture all grammatical or semantic distinctions.



Figure 4.8. Word clouds showing the most frequent words in two texts. Their sizes indicate relative frequencies; these are illustrations rather than a complete numerical histogram.

Further reading. For the theory, see [39, 46]. Computational methods and applications are developed in [35]. The companion course Optimal Transport for Machine Learning (OT4ML) offers another route into these ideas.

CHAPTER 5

Neural networks

Neural networks learn transformations from examples. This chapter explains how layers are composed and trained, then shows how a generator can learn to reproduce a data distribution. The connection with optimal transport clarifies both the learning objective and its limitations.

5.1 Discriminative networks

5.1.1 A function built from layers

A neural network is a parameterized function f_w. Given an input x, such as an image, it computes an output $\hat{y} = f_w(x)$. For classification, the output can represent estimated probabilities of several classes. A large output probability expresses the model's confidence; it does not by itself ensure that this confidence is well calibrated or that the prediction is correct.

To begin, suppose an image belongs to exactly one of two classes, cat or dog. A classifier can return two nonnegative numbers summing to one. This differs from a multilabel task, where an image may contain both a cat and a dog: in that case, separate probabilities need not sum to one.

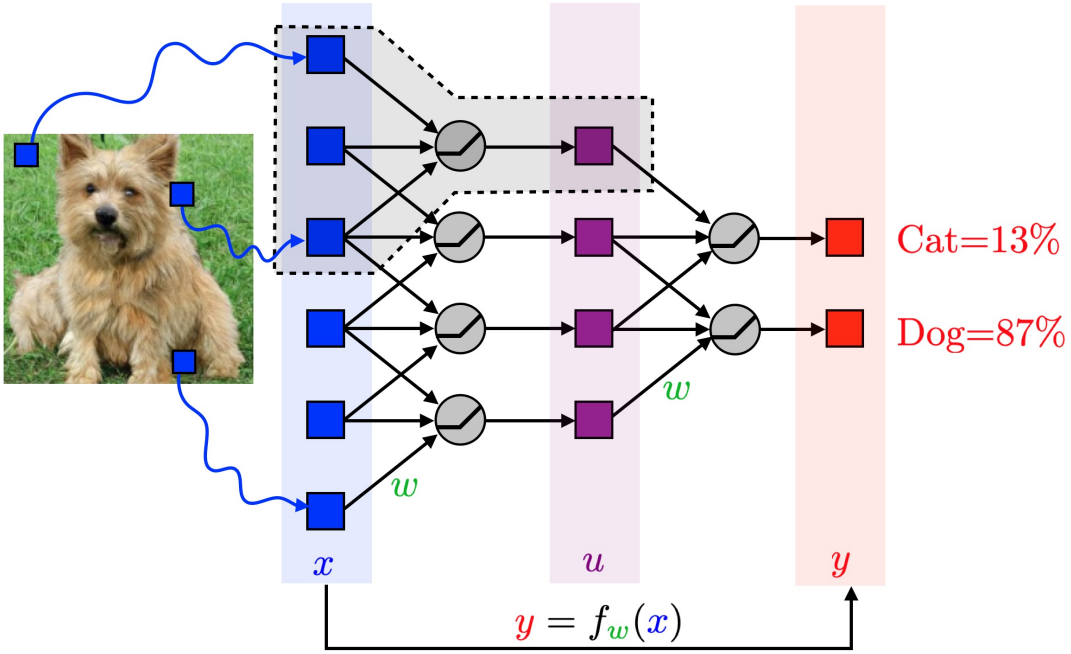


Figure 5.1. A schematic classifier with one hidden layer and one output layer. Only a few input pixels and connections are drawn. The displayed percentages are model predictions; a normalization such as softmax is needed to obtain class probabilities.

Each layer combines affine operations with an activation function. A rectified linear unit, or ReLU, computes

$$u_1 = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + w_3 x_3 + b), \qquad \mathrm{ReLU}(t) = \max(t, 0). \tag{5.1}$$

The weights determine the contribution of each input; the bias b shifts the activation threshold. A ReLU outputs zero for a nonpositive input and passes positive inputs unchanged. It differs from a binary threshold neuron, which has only two possible output values.

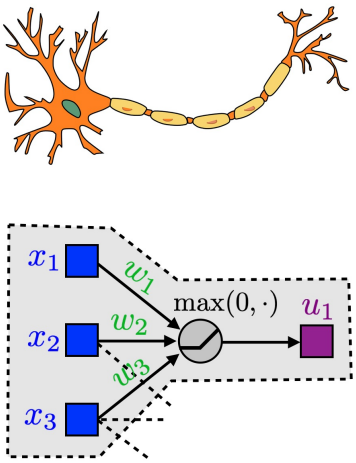


Figure 5.2. A biological neuron and a schematic artificial neuron. The mathematical model keeps only a weighted sum and an activation; the bias in (5.1) is omitted from the drawing.

For an input $x \in \mathbb{R}^d$, a hidden layer of width m, and K classes, one possible network is

$$u = \mathrm{ReLU}(Wx + b), \qquad a = Vu + c, \qquad \hat{y}_k = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}},$$

where $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, $V \in \mathbb{R}^{K \times m}$, and $c \in \mathbb{R}^K$. ReLU acts coordinatewise. The final operation is softmax; the intermediate values a_k are called logits. In computation, replacing every a_k by $a_k - \max_j a_j$ gives the same probabilities and avoids overflow. All entries of W, b, V, c form the parameter vector w.

Composing more layers gives a deep network. Nonlinear activations are essential: a composition of affine maps is still affine. Depth can represent certain functions more economically than a very wide single hidden layer, but the appropriate architecture depends on the task.

The biological terminology is an analogy, not a faithful model of the brain. McCulloch and Pitts studied logical threshold neurons [31], while Rosenblatt's perceptron introduced an early learning model [37]. These models should be distinguished from the ReLU networks described here. Biological studies of the visual cortex and of membrane dynamics concern much richer mechanisms [20, 22].

5.1.2 Learning from labeled examples

Supervised learning starts from pairs $(x_i, y_i)_{i=1}^n$. For a single class label k_i, the target is often a one-hot vector: $(y_i)_{k_i} = 1$, with all other entries zero. Such labels record annotations, rather than known conditional probabilities. The data may come from human annotation or another source of supervision, and may contain mistakes.



Figure 5.3. Examples of labeled dog and cat images from ImageNet [15]. Training uses many examples; evaluation requires separate data.

A training objective measures disagreement between predictions and labels. Two examples are the mean squared error and the cross-entropy loss:

$$E_{\mathrm{sq}}(w) = \frac{1}{n}\sum_{i=1}^{n} \|f_w(x_i) - y_i\|_2^2, \tag{5.2}$$

$$E_{\mathrm{CE}}(w) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} (y_i)_k \log(f_w(x_i)_k). \tag{5.3}$$

For a one-hot target, the corresponding cross-entropy term is $-\log(f_w(x_i)_{k_i})$. The logarithm is natural, and softmax gives strictly positive probabilities for finite logits. A stable implementation evaluates the log-softmax directly. A penalty on the weights can be added to the objective.

Training seeks to reduce this empirical loss. Because the network depends nonlinearly on many parameters, the objective is generally nonconvex. Optimization methods can work well without providing a general guarantee of reaching its global minimum.

5.1.3 Backpropagation and weight updates

Backpropagation computes the gradient of a scalar loss by applying the chain rule through the network in reverse order [28, 38]. It is reverse-mode automatic differentiation. An optimizer then uses these derivatives to update the weights; for example, gradient descent takes the step

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla E(w^{(t)}), \qquad \eta_t > 0.$$

The step size matters: an excessively large step can increase the loss. Stochastic gradient descent replaces the full gradient by a gradient estimated from a randomly chosen minibatch, reducing the cost of each update. An individual stochastic step need not decrease the full training loss.

For the two-layer classifier above and one example with cross-entropy ℓ, the chain rule is particularly transparent. Write $r = Wx + b$, $u = \mathrm{ReLU}(r)$, and $\delta = \hat{y} - y$. Away from $r_j = 0$,

$$\frac{\partial \ell}{\partial V} = \delta u^{\mathsf{T}}, \qquad \frac{\partial \ell}{\partial c} = \delta, \qquad q = (V^{\mathsf{T}}\delta) \odot \mathbf{1}_{r>0}, \qquad \frac{\partial \ell}{\partial W} = q x^{\mathsf{T}}, \quad \frac{\partial \ell}{\partial b} = q.$$

Here $\odot$ is coordinatewise multiplication. At a ReLU kink, software chooses a convention, commonly zero, for the derivative of the activation. That convention does not make the network classically differentiable at the kink.

Large labeled datasets, efficient GPU computation, and improvements in architectures and training contributed to the success of deep image classifiers, notably the 2012 ImageNet result [25]. Regularization techniques such as dropout can help limit overfitting [43]. The work of Yoshua Bengio, Geoffrey Hinton, and Yann LeCun was recognized by the 2018 ACM A. M. Turing Award, announced in 2019.[1]

5.1.4 Approximation, optimization, and generalization

A universal approximation theorem concerns what a network can represent. A classical result of Cybenko [13] states that, for a continuous function $F : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exist a finite m and coefficients such that

$$G(x) = \sum_{j=1}^{m} \alpha_j \sigma(v_j \cdot x + \beta_j), \qquad \sup_{x\in[0,1]^d} |F(x) - G(x)| < \varepsilon,$$

where σ is a fixed continuous sigmoidal activation, such as $\sigma(t) = 1/(1 + e^{-t})$. The output layer is linear, and the width and weights are unrestricted. This is the sigmoidal theorem; approximation results for ReLU networks require their own statements.

The theorem does not involve training data. It neither requires infinitely many examples nor guarantees that a training algorithm can find the approximating network from finitely many observations. Three questions must be separated: whether a good representation exists, whether optimization can find it, and whether the learned predictor performs well on new data.

Generalization is evaluated on data excluded from training. Validation data guide choices such as architecture and stopping time; a separate test set estimates the performance of the final procedure. Good performance also depends on whether these data represent the conditions in which the network will be used.

[1] https://awards.acm.org/about/2018-turing

Convolutional networks build in assumptions about spatial structure [26]. Local connections restrict the receptive field, and weight sharing applies the same filter at different positions. A stride-one convolution commutes with translations on an infinite or periodic grid; boundary handling and subsampling can change this property. Translation equivariance of a feature map should also be distinguished from translation invariance of a final classification.

Architecture, regularization, data, and optimization all influence performance [27]. Mathematical results explain particular mechanisms and regimes, while a complete account of every successful architecture remains beyond a single theorem.

5.2 Generative networks

5.2.1 From random inputs to a distribution

A generator g_w transforms a random input $Y \in \mathbb{R}^r$ into an image $X = g_w(Y) \in \mathbb{R}^d$. The input law η, for instance a standard Gaussian, is chosen in advance. The output law is the pushforward $\mu_w = (g_w)_\# \eta$. This notation describes how probability mass is redistributed by the network. A generator is a learned transformation, not generally the inverse of a classifier.

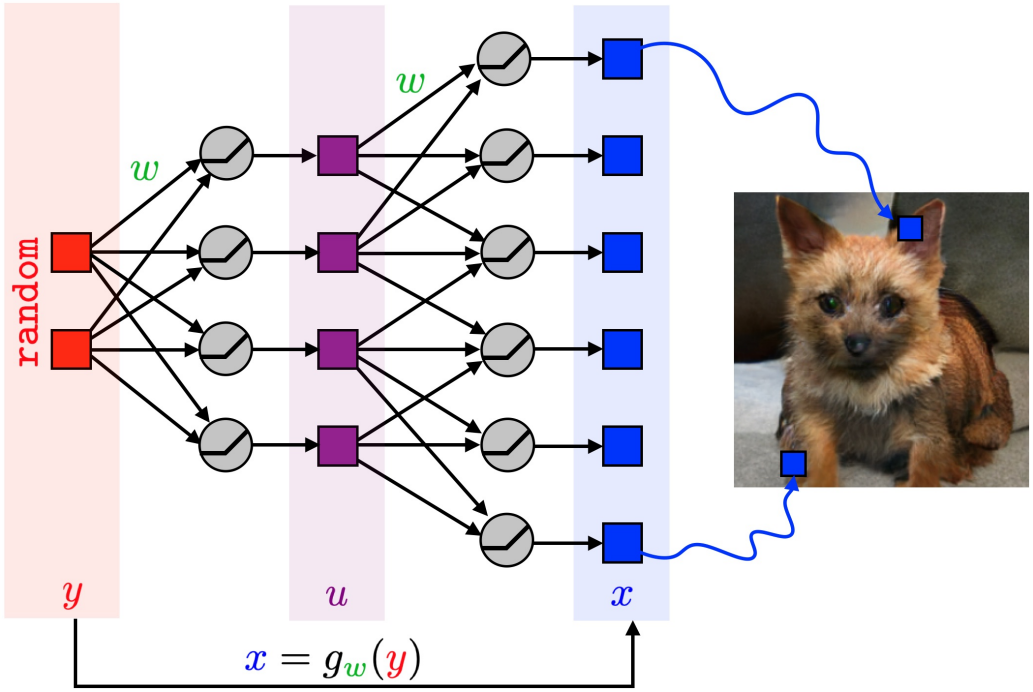


Figure 5.4. A schematic generator maps a random input y through a hidden layer to an image $x = g_w(y)$. Practical image generators have more layers and a much larger output dimension.

For unconditional generation, training can use images without class labels. Their collection, selection, and preprocessing still require care. Conditional generation additionally supplies information such as a class label, giving a model $g_w(y, c)$. Thus not every generative model is trained without supervision.

The aim is to reproduce a distribution, including its diversity. If the reference distribution assigns probability $1/2$ to cats and $1/2$ to dogs, an ideal generator has those probabilities too. A finite random batch need not contain exactly half of each. A model that produces the same plausible image repeatedly has not learned the full distribution; this is an extreme form of mode collapse.

5.2.2 Comparing empirical distributions

Let $z_1, \dots, z_n$ be training images, and let $x_i = g_w(y_i)$ be generated images from independent latent samples. Their empirical distributions are

$$\mu_{w,n} = \frac{1}{n} \sum_{i=1}^{n} \delta_{x_i}, \qquad \nu_n = \frac{1}{n} \sum_{j=1}^{n} \delta_{z_j}.$$

Using measures retains frequencies and repeated samples, which ordinary set notation would discard. One possible objective is a transport distance between these measures.

For Euclidean ground cost, the empirical Wasserstein distance of order one is

$$W_1(\mu_{w,n}, \nu_n) = \frac{1}{n} \min_{s \in \Sigma_n} \sum_{i=1}^{n} \|g_w(y_i) - z_{s(i)}\|_2. \tag{5.4}$$

The factor $1/n$ normalizes the total probability mass. This distance is different from the squared quadratic cost introduced in the transport chapter. For unequal sample counts or weights, replace the permutation by a coupling with the prescribed marginals.

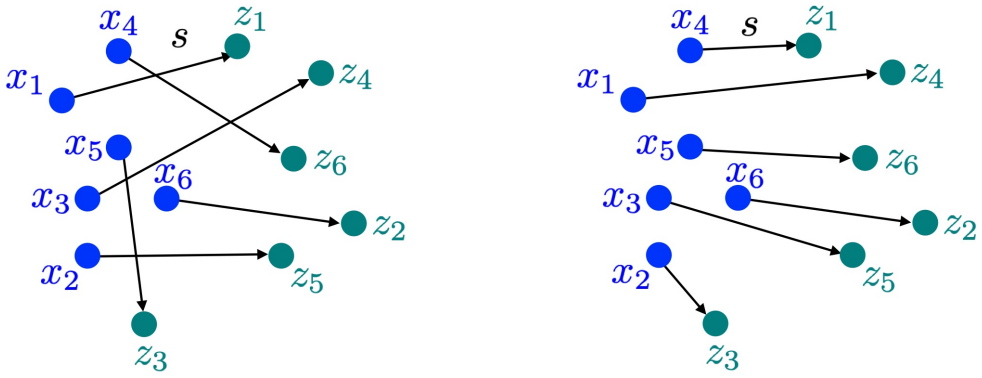


Figure 5.5. Two assignments of six source points to six target points. The right-hand assignment has smaller Euclidean transport cost and is the optimal matching in this example.

In Figure 5.5, the left assignment is $(1, 5, 4, 6, 3, 2)$, while the optimal one on the right is $(4, 3, 5, 1, 6, 2)$. The notation lists $(s(1), \dots, s(6))$. Although there are $6! = 720$ possibilities, efficient assignment algorithms avoid enumerating all $n!$ permutations and can solve a dense problem in $O(n^3)$ operations. The transport chapter explains the convex relaxation behind this computation [35].

The distributional learning objective is $\min_w W_1(\mu_w, \nu)$ for a target law ν. Finite-sample transport provides a numerical approximation, but brings statistical and optimization difficulties. Transport between small batches is generally a biased estimate of the population distance. The optimal assignment may change as w changes, so the empirical objective is not everywhere differentiable.

The ground distance also matters. Euclidean distance between pixel arrays is sensitive to small translations and may poorly reflect semantic similarity. A feature representation can provide a more relevant cost, but its choice changes the geometry being learned.

5.2.3 Generative adversarial networks

Generative adversarial networks (GANs), introduced by Goodfellow and collaborators in 2014 [19], train a generator against a discriminator. The discriminator $D_v(x) \in (0, 1)$ tries to distinguish data from generated samples. The original minimax objective is

$$\min_w \max_v \left\{ \mathbb{E}_{Z \sim \nu}[\log D_v(Z)] + \mathbb{E}_{Y \sim \eta}[\log(1 - D_v(g_w(Y)))] \right\}. \tag{5.5}$$

The discriminator increases the objective; the generator decreases it. Alternating gradient updates approximate this game, without a general guarantee of convergence for arbitrary networks.

For an unrestricted discriminator optimized exactly, the inner supremum equals $-\log 4 + 2\,\mathrm{JS}(\nu, \mu_w)$, where the Jensen–Shannon divergence is

$$\mathrm{JS}(\nu, \mu) = \tfrac{1}{2}\,\mathrm{KL}(\nu \mid m) + \tfrac{1}{2}\,\mathrm{KL}(\mu \mid m), \qquad m = \tfrac{1}{2}(\nu + \mu).$$

Here KL uses natural logarithms and is defined between measures by their Radon–Nikodym densities. Both measures are absolutely continuous with respect to m. This identity concerns the ideal discriminator optimum; finite networks and incomplete optimization change the objective actually achieved. In practice, a commonly used generator loss is $-\mathbb{E} \log D_v(g_w(Y))$, which has the same desired distributional optimum but different gradients.

5.2.4 The Wasserstein connection

Wasserstein GANs [3] use a different objective, based on the Kantorovich–Rubinstein dual formula. For probability measures on $\mathbb{R}^d$ with finite first moments,

$$W_1(\nu, \mu_w) = \sup_{\mathrm{Lip}(h)\le 1} \left\{ \mathbb{E}_{Z\sim\nu} h(Z) - \mathbb{E}_{Y\sim\eta} h(g_w(Y)) \right\}. \tag{5.6}$$

The constraint means $|h(x) - h(x')| \le \|x - x'\|_2$ for all x, x'. A critic network approximates this class of functions, and its values are real scores, not class probabilities. The generator minimizes the discrepancy estimated by the critic.

The critic is a dual potential for a *fixed* ground distance; it does not automatically learn a new transport cost between pairs of images. Enforcing or approximating the Lipschitz constraint is essential. Restricting the critic class and optimizing it only approximately also means that the computed score need not equal the true Wasserstein distance.

Ordinary GANs and Wasserstein GANs thus express related adversarial principles with different mathematical objectives. Neither training procedure guarantees that every generated image is meaningful or that all modes are represented.



Figure 5.6. Two sequences of synthetic images interpolating between cats and dogs, using the BigGAN model [6]. BigGAN is class-conditional; its class information is part of the conditioning. These examples illustrate generation and interpolation rather than a claim of an optimal transport path.

Interpolating latent or conditioning variables can produce smooth visual transitions, as in Figure 5.6. Such paths need not be transport geodesics or remain typical under the training distribution. Visual plausibility, diversity, fidelity to the target distribution, and performance on held-out data are separate aspects of a generative model's quality.

Glossary

Terms introduced in the first two chapters, collected here for reference.

Information and coding

- **Pixel**: a location in an image grid, or its stored value.
- **Symbol**: an element of the finite alphabet to be encoded.
- **Codeword**: the bit string assigned to one symbol.
- **Code**: the assignment of codewords to symbols.
- **Empirical distribution**: the observed relative frequencies of symbols.
- **Histogram**: a graphical representation of these frequencies.
- **Source model**: a probability law for the symbols or their sequences.
- **Entropy**: the nonnegative average information $H(V)$, in bits.
- **Average code length**: $L(c) = \sum_v p_v \ell_v$, in bits per symbol.

Image processing

- **Bit and byte**: a binary digit; a group of eight bits.
- **Channel**: one component array of a multichannel image.
- **Quantization**: replacing values by indices from a finite set.
- **Noise**: unwanted fluctuations introduced by acquisition or processing.
- **Mean**: the sum of values divided by their number.
- **Median**: the central sorted value for an odd-sized set.
- **Edge**: a rapid spatial change in image values.
- **Blur**: a loss of sharp transitions and fine detail.
- **Contrast**: differences between light and dark regions.
- **Luminance**: a photometric quantity defined using visual sensitivity.
- **Matrix**: a rectangular array $(a_{i,j})$.
- **Resolution**: a term that may refer to pixel dimensions or to spatial resolving power; these are distinct notions.
- **Crossfade**: a weighted average of two images at fixed pixel locations.

References

[1] B. Adcock, A. C. Hansen, C. Poon, and B. Roman. Breaking the coherence barrier: asymptotic incoherence and asymptotic sparsity in compressed sensing. *CoRR*, abs/1302.0561, 2013. URL http://arxiv.org/abs/1302.0561.

[2] Martial Agueh and Guillaume Carlier. Barycenters in the wasserstein space. *SIAM Journal on Mathematical Analysis*, 43(2):904–924, 2011.

[3] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223, 2017. URL https://proceedings.mlr.press/v70/arjovsky17a.html.

[4] Garrett Birkhoff. Tres observaciones sobre el álgebra lineal. *Universidad Nacional de Tucumán Revista Series A*, 5:147–151, 1946.

[5] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991.

[6] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1809.11096.

[7] E. J. Candès, J. Romberg, and T. Tao. Stable signal recovery from incomplete and inaccurate measurements. *Communications on Pure and Applied Mathematics*, 59(8):1207–1223, 2006.

[8] E.J. Candès and C. Fernandez-Granda. Towards a mathematical theory of super-resolution. *Communications on Pure and Applied Mathematics*, 67(6):906–956, 2014. ISSN 1097-0312. doi: 10.1002/cpa.21455. URL http://dx.doi.org/10.1002/cpa.21455.

[9] N. Chauffert, P. Ciuciu, J. Kahn, and P. Weiss. Variable density sampling with continuous trajectories. *SIAM Journal on Imaging Sciences*, 7(4):1962–1992, 2014.

[10] S. S. Chen, D. L. Donoho, and M. A. Saunders. Atomic decomposition by basis pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010.

[11] Patrick L. Combettes and Jean-Christophe Pesquet. Proximal splitting methods in signal processing. In *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*, pages 185–212. Springer, 2011. doi: 10.1007/978-1-4419-9569-8_10.

[12] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.

[13] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[14] George B. Dantzig. Application of the simplex method to a transportation problem. In Tjalling C. Koopmans, editor, *Activity Analysis of Production and Allocation*, pages 359–373. Wiley, 1951.

[15] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[16] D. L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.

[17] M. F. Duarte, M. A. Davenport, D. Takhar, J. N. Laska, T. Sun, K. F. Kelly, and R. G. Baraniuk. Single-pixel imaging via compressive sampling. *IEEE Signal Processing Magazine*, 25(2):83–91, March 2008. ISSN 1053-5888.

[18] V. Duval and G. Peyré. Exact support recovery for sparse spikes deconvolution. *Foundations of Computational Mathematics*, 15(5):1315–1355, 2015. doi: 10.1007/s10208-014-9228-6. URL http://hal.archives-ouvertes.fr/hal-00839635/.

[19] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[20] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*, 117(4):500–544, 1952.

[21] Gao Huang, Chuan Guo, Matt J Kusner, Yu Sun, Fei Sha, and Kilian Q Weinberger. Supervised word mover's distance. In *Advances in Neural Information Processing Systems*, pages 4862–4870, 2016.

[22] David H Hubel and Torsten N Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of physiology*, 160(1):106–154, 1962.

[23] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the Institute of Radio Engineers*, 40(9):1098–1101, 1952.

[24] Leonid Kantorovich. On the transfer of masses (in russian). *Doklady Akademii Nauk*, 37(2):227–229, 1942.

[25] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.

[26] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[27] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[28] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16 (2):146–160, 1976.

[29] S. G. Mallat. *A Wavelet Tour of Signal Processing: The Sparse Way*. Elsevier/Academic Press, Amsterdam, third edition, 2009.

[30] S. G. Mallat and Z. Zhang. Matching pursuits with time-frequency dictionaries. *IEEE Transactions on Signal Processing*, 41(12):3397–3415, 1993.

[31] Warren S McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[32] Gaspard Monge. Mémoire sur la théorie des déblais et des remblais. *Histoire de l'Académie Royale des Sciences*, pages 666–704, 1781.

[33] B. K. Natarajan. Sparse approximate solutions to linear systems. *SIAM Journal on Computing*, 24(2): 227–234, 1995. doi: 10.1137/S0097539792240406.

[34] Gabriel Peyré. The numerical tours of signal processing. *Computing in Science & Engineering*, 13(4):94–97, 2011. doi: 10.1109/MCSE.2011.71. URL https://www.numerical-tours.com/.

[35] Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. URL https://arxiv.org/abs/1803.00567.

[36] J. Rissanen and G. Langdon. Arithmetic coding. *IBM Journal of Research and Development*, 23(2):149–162, 1979.

[37] Frank Rosenblatt. The perceptron: A perceiving and recognizing automaton. Technical Report 85-460-1, Cornell Aeronautical Laboratory, 1957.

[38] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[39] Filippo Santambrogio. *Optimal transport for applied mathematicians*. Birkhäuser, 2015.

[40] F. Santosa and W.W. Symes. Linear inversion of band-limited reflection seismograms. *SIAM Journal on Scientific and Statistical Computing*, 7(4):1307–1330, 1986. doi: 10.1137/0907087.

[41] C. E. Shannon. A Mathematical Theory of Communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[42] C. E. Shannon. Communication in the presence of noise. *Proc. Institute of Radio Engineers*, 37(1):10–21, 1949.

[43] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1): 1929–1958, 2014.

[44] R. Tibshirani. Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society. Series B. Methodological*, 58(1):267–288, 1996.

[45] Samuel Vaiter, Gabriel Peyré, and Jalal Fadili. Low complexity regularization of linear inverse problems. In Götz E. Pfander, editor, *Sampling Theory, a Renaissance*, pages 103–153. Birkhäuser, 2015. URL https://hal.science/hal-01018927.

[46] Cédric Villani. *Topics in Optimal Transportation*. Graduate Studies in Mathematics Series. American Mathematical Society, 2003. ISBN 9780821833124.

[47] John Von Neumann. A certain zero-sum two-person game equivalent to the optimal assignment problem. *Contributions to the Theory of Games*, 2:5–12, 1953.